UNIT 1

ABSTRACTION: in C++ means **showing only essential details** and **hiding unnecessary implementation details** from the user.
It helps to focus on *what an object does*, rather than *how it does it*.

Abstraction is Achieved in C++ by two ways:

- **Using Classes:**
Hide data and implementation using `private` and `protected` access specifiers.
Example: Private members are hidden from outside.

- **Using Abstract Classes (with pure virtual functions):**
Define only the interface, and leave implementation to derived classes.

ABSTRACT CLASS CODE:

```
#include <iostream>

using namespace std;


class Shape {

public:

   // Pure virtual function (no definition)

   virtual void draw() = 0;

};


class Circle : public Shape {

public:

   void draw()        //override

{

       cout << "Drawing Circle" << endl;

   }
```

OOPS WITH C++

```
};

int main() {
    Shape* s = new Circle();                //here circle() pointing object
    s->draw();
    return 0;
}
```

-> Here ,Circle provides the actual implementation.

The user just calls draw() — doesn't need to know how the drawing is done internally.

*CODE TO IMPLEMENT ABSTRACTION(a user can only start the car without thinking about the internal implementation details)

Encapsulation: Encapsulation means wrapping (hiding) data and code together into a single unit (class).
It also means protecting data from direct access — only allowing it through functions (methods).

CODE:
```
#include <iostream>

using namespace std;

class Student {

private:
    // data members are hidden (cannot be accessed directly)
```

OOPS WITH C++

```
    string name;
    int age;

public:
    // function to set data (setter)
    void setData(string n, int a) {
        name = n;
        age = a;
    }

    // function to display data (getter)
    void showData() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

int main() {
    Student s1;

    // set data using public function
    s1.setData("Saba", 22);
```

OOPS WITH C++

```
    // display data using public function
    s1.showData();


    return 0;


}
```

OOPS WITH C++

1. Meaning of Object Orientation

Object Orientation means thinking and designing a system around **objects** instead of just functions or logic.

- An **object** represents a **real-world entity** — like a student, car, or bank account.
- Each object has:
  - **Data (attributes)** — like name, age, balance
  - **Functions (methods)** — like deposit(), display(), drive()

Object orientation is organizing code around *objects* that combine **data + behavior**.

2. Object Identity

Each object has its **own unique identity**, even if two objects have the same data.
Example:

```
Student s1("Saba");

Student s2("Saba");
```

Both have the same name, but they are **different objects** — like two students with the same name but different roll numbers.

Every object is unique and has its own place in memory which is called object identity.

3. Encapsulation

Encapsulation means **wrapping data and functions together** inside a class so they work as one unit.

- Data is **protected** from direct access.
- You can only change it through specific functions (getters/setters).

Example:

class Student { private: int marks; public: void setMarks(int m) { marks = m; } int getMarks() { return marks; } };

4. Information Hiding(ABSTRACTION)

This means **keeping internal details private** and showing only what's necessary to the user.
Example: When you use a phone, you press buttons but don't see how circuits work inside.

In programming:

- You hide data using private or protected keywords.
- You show only necessary functions as public.

Information hiding keeps the internal working of a class hidden from the outside world.

5. Polymorphism

Polymorphism means **one name — many forms**.
It allows the same function to behave differently based on the object that calls it.

6. Generality (Generosity)

OOPS WITH C++

Generality (sometimes called **Generosity**) means designing systems so that **one solution works for many cases** — not just one.
Example:

- Writing a generic "sort" function that can sort **integers, floats, or strings**.
- Using **templates** in C++ for reusability.

Generality means Make code flexible and reusable.

7. Importance of Modelling

Modeling means creating a **simplified representation** of a real system before building it.
Like drawing a **blueprint** before constructing a house.

Importance Of Modeling:

- Helps understand the system clearly
- Reduces mistakes
- Saves time and cost
- Makes communication easy among developers

Modeling helps visualize and plan a system before coding it.

8. Principles of Modelling

Good models follow certain **principles**:

1. The model must represent reality correctly.
2. The model must be understandable.
3. The model should be easily changeable.

A good model is simple, accurate, and easy to modify.

9. Object Oriented Modelling

OOPS WITH C++

It is the process of **analyzing and designing a system using object-oriented concepts** (like class, object, inheritance, etc.).

Steps:

1. Identify **objects** in the system
2. Define their **attributes** and **methods**
3. Define **relationships** between objects

Object-Oriented Modelling is Representing a system using interacting objects.

Introduction to UML (Unified Modeling Language)

UML is a **visual language** used to **design and document** object-oriented systems.
It has **diagrams** to show different aspects of the system like:

- Class Diagram (structure)
- Use Case Diagram (user interaction)
- Sequence Diagram (object communication)
- Activity Diagram (workflow)

It is a set of diagrams used to visualize, design, and document software systems.

OOPS WITH C++

Conceptual Model of UML

The **conceptual model** of UML is built on 3 main concepts:

1. **Things** — building blocks (classes, objects, components, nodes)
2. **Relationships** — how things are connected (association, inheritance, etc.)
3. **Diagrams** — how you visualize the system (class, use case, sequence, etc.)

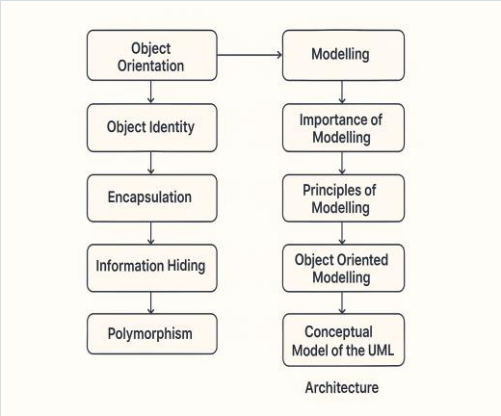
UML's conceptual model explains *what elements exist, how they relate, and how they are visualized.*

Architecture of UML

UML has a **five-view architecture** (based on the 4+1 model):

1. **Use Case View** — what users want (functionalities)
2. **Design View** — structure of the system (classes, objects)
3. **Process View** — system's runtime behavior (threads, processes)
4. **Implementation View** — how the system is built (components)
5. **Deployment View** — how it is deployed on hardware

UML Architecture is a 5 views that describe the system from different perspectives.



UML (Unified Modeling Language)

UML is a **visual language** used to **design, visualize, and document** software systems — especially **object-oriented** ones.

It contains **two main types of diagrams**:

1. Structural Diagrams (Show System Structure)

These diagrams show **how the system is built** — the classes, objects, and relationships.

a) Class Diagram

Shows **classes**, their **attributes**, **methods**, and **relationships**.

Example:

```
+-----------------+
|  Student        |
+-----------------+
| - name          |
```

OOPS WITH C++

| - rollNo |

+----------+

| + study() |

| + showDetails() |

+----------+

|

| 1..*

|

+----------+

| Course |

+----------+

| - courseName |

| - courseCode |

+----------+

| + enroll() |

+----------+

b) Object Diagram

Shows **real objects** of classes and their links.

Student1 : Student

name = "Saba"

rollNo = 101

Course1 : Course

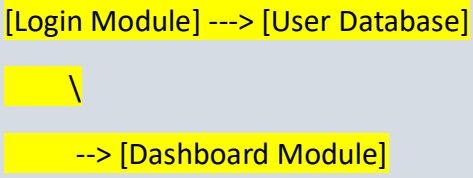
courseName = "DBMS"

Student1 --> Course1

OOPS WITH C++

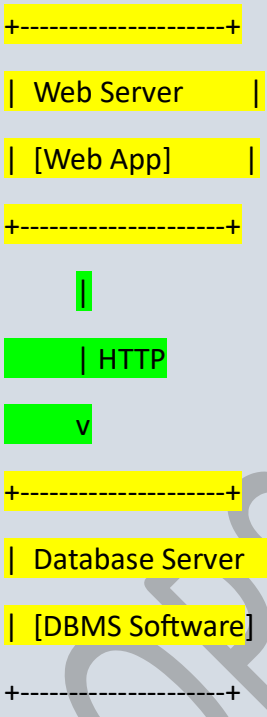
c) Component Diagram

Shows how the system is **divided into components** (modules, files, etc.)



d) Deployment Diagram

Shows **hardware and software** deployment.

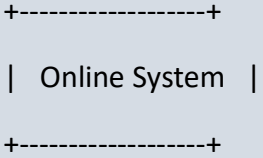


2. Behavioral Diagrams (Show System Behavior)

These show **how objects interact and behave over time**.

a) Use Case Diagram

Shows **user (actor)** interactions with the system.



OOPS WITH C++

/ | \

/ | \

[Register] [Login] [Purchase Item]

^

|

User

Here, User can register, login, and purchase.

b) Sequence Diagram

Shows **how objects interact step-by-step**.

User LoginPage Server

| | |

| --enter()--> | | ← User enters login details

| | --verify()--> | ← LoginPage sends data to Server

| | <--result--- | ← Server sends verification result

| <--response--| | ← LoginPage sends final response to user

c) Activity Diagram

Shows the **flow of control (like a flowchart)**.

Start

|

[Enter Login Info]

|

[Verify Credentials]

|---Yes--->[Dashboard]

|---No---->[Show Error]

|

End

OOPS WITH C++

d) State Diagram

Shows **different states of an object**.

[Idle] ---> [Processing] ---> [Completed]

<--- [Cancelled]

NOTES BY,

MS. Saba Nehal

OOPS WITH C++

UNIT 2

1. Basic Structural Modeling

Structural modeling is about **describing the static structure** of a system—how different parts of a system are related to each other. In object-oriented programming (like C++), this includes **classes**, **objects**, and their **relationships**.

Classes

- A **class** is like a **blueprint** for creating objects.
- It defines **properties (data members)** and **behaviors (member functions)**.
- Example:

```
class Car {
  public:
    string color;
    void drive() {
       cout << "Driving...";
    }
};
```

Relationships

Relationships define **how classes or objects are connected** to each other. There are several types:

1. **Association**: A general connection between classes (e.g., a teacher "teaches" a student).
2. **Aggregation**: A "whole-part" relationship where the part can exist independently (e.g., a library has books).
3. **Composition**: A stronger "whole-part" relationship; if the whole is destroyed, the part is also destroyed (e.g., a house has rooms).
4. **Inheritance**: One class (child) inherits from another (parent), reusing its properties and behaviors.
5. **Dependency**: One class depends on another temporarily (e.g., using an object of another class in a method).

by saba nehal

Common Mechanisms

These are features commonly used across different models:

- **Abstraction**: Hiding details and showing only the necessary features.
- **Encapsulation**: Wrapping data and functions together.
- **Modularity**: Breaking a system into smaller parts (modules).
- **Hierarchy**: Organizing components into levels (like parent-child in inheritance).

Diagrams

Structural modeling uses diagrams to **visually represent** the system's structure:

- **Class Diagrams**
- **Object Diagrams**
- **Component Diagrams**
- **Deployment Diagrams**

Class & Object Diagrams

Class Diagram

- Shows the **classes in a system** and their **relationships**.
- Used to design or understand the **static structure** of a system.

Class Diagram Components:

1. **Class Name**
2. **Attributes** (data members)
3. **Operations** (methods)
4. **Visibility**: + (public), - (private), # (protected)

Example:

```
+----------------------+
|      Car             |
+----------------------+
| - color: string      |
| - speed: int         |
+----------------------+
```

by saba nehal

| + drive(): void |

+----------------------+

Relationships shown in Class Diagrams:

- Inheritance (Generalization)
- Association
- Aggregation
- Composition
- Dependency

Object Diagram

- Shows a **snapshot** of objects at a specific moment during program execution.
- It's like a real-world version of a class diagram with actual **values** instead of just definitions.

Example:

If you have a class Car, then an object diagram might show:

car1:Car

color = "Red"

speed = 100

So it visualizes:

- Specific **instances (objects)**
- **Values of attributes**
- **Links** between objects (like how they interact or relate)

by saba nehal

Collaboration Diagrams (also called Communication Diagrams)

Collaboration Diagrams show **how objects interact with each other** and **how messages are passed between them**, **focusing on the structure and relationships** between objects.

It focuses on:

1.OBJECTS INVOLVED: (instances of classes)

2.LINKS:(relationship between objects **)**

3.MESSAGES: (interaction between objects)

4.MESSAGE SEQUENCE NUMBER: (order of execution)

WHERE WE USED COLLABORATION DIAGRAM?

1. In object oriented design.
2. Understand how C++ objects iteract during execution.
3. Visualizing function calls between objects.

Features:

showObjects : An instance of a class. Shown as a rectangle with the object name and class (like :Customer).

- **showsLinks**: A line connecting two objects shows that they communicate.
- **showsMessages**: Messages are labelled on the line[method calls] (like 1: placeOrder()).
- **Shows sequence numbers**: Order of messages is shown like 1,1.1,1.1.1,2,…..

This diagram shows **which objects interact** and **in what order** messages are passed.

Depicting a Message: visual representation of a function call(method call) sent from one object to another,showing who sends the message ,to whom ,and in what order.

- Messages are shown as **numbered arrows** (like 1: calculateTotal()).
- They show **what action** one object asks another to perform.
- Sequence numbers (1, 1.1, 2, etc.) show **the order** in which messages occur.

Polymorphism in Collaboration Diagrams:

Polymorphism means different objects can respond to the same message in different ways.

- In collaboration diagrams, the **same message** (e.g., draw()) can be sent to different objects (e.g., Circle, Square), and **each object responds differently** based on its class.
- This helps represent **runtime behavior** in an object-oriented system.

Iterated Messages:

- When a message is sent **multiple times**, like in a loop.

by saba nehal

- Its purpose not just who sends the message but also how many times or under which condition the message is repeated.
- Shown as:
  * [condition] messageName(),
  Example: * [for each item] calculatePrice()
- It means this message repeats **for each item**, or **as long as a condition is true**.

Use of Self in Messages:

- Sometimes, an object may **send a message to itself**.
- This shows **internal behavior** or when a method calls another method of the same object.
- Represented by an arrow looping back to the same object.

Example:
Object1 -→ Object1 : methodName()

ADVANTAGES OF USING COLLABORATION DIAGRAM

1.SHOWS RELATIONSHIP CLEARLY: Easy to see how objects are linked clearly.

2.USEFUL FOR CODE UNDERSTANDING: We can see method call flow.

3. HELPS IN DESIGNING: Before writing code, we visualize interaction.

DIAGRAMS 1. Student requesting a book from library.

2.Online shopping(add item to cart)

3.Hospital appointment booking.

Sequence Diagrams

Sequence diagrams focus on **time-based behavior** — how objects interact **over time** by exchanging messages.

it shows **who talks to whom**, **what they say**, and **in what order**.

by saba nehal

Need of Sequence Diagram:

- To show how a particular task happens step-by-step.
- To visualize the flow of messages between system components.
- To help understand or design logic of a process (e.g., login, order processing, etc.)

Elements of a Sequence Diagram

1. Actor	The user or external system that interacts with the system (e.g., Customer).
2. Object / Class	The components of the system that send/receive messages .
3. Lifeline	A dashed vertical line that shows the existence of an object over time.
4. Activation box	A small rectangle on the lifeline — shows when an object is active (doing some work).
5. Message (Arrow)	Communication between objects (method call, response, etc.).
6. Return Message	A dashed arrow that shows a response or result.

FOR Example: User Login Process

Actors and objects:

- User
- Login Page
- Server
- Database

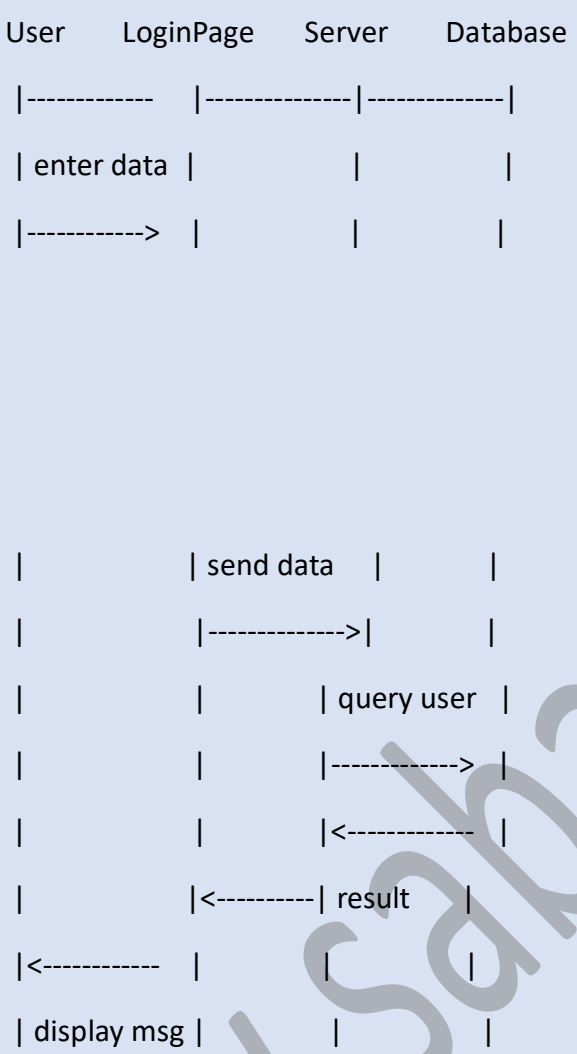
WORKFLOW:

1. **User** enters username and password on the **Login Page**.
2. **Login Page** sends credentials to the **Server**.
3. **Server** sends query to the **Database** to check credentials.
4. **Database** returns validation result to the **Server**.

by saba nehal

5. **Server** sends login success/failure message back to **Login Page**.
6. **Login Page** shows the result to the **User**.

SEQUENCE DIAGRAM OF THE ABOVE CASE



DIAGRAMS: 1. Online shopping (add to cart, checkout)

2. ATM transaction

3. Payment processing

4. Chat message delivery

Depicting Asynchronous Messages (With / Without Priority):

- **Asynchronous messages** are when the sender sends the message and **does not wait** for the receiver to finish.

by saba nehal

- Shown with a **solid line and open arrowhead**.
- Priority (optional): Can show **order of preference** or **urgency**, but UML doesn't define strict priority syntax. You can label messages like:
    - <<high>> notifyUser()
    - <<low>> logError()

Call-back Mechanism:

- A **callback** happens when an object **sends a message**, and the receiver **calls back later**, often via a function pointer, interface, or lambda.
- In sequence diagrams:
    - You show the **initial call**, then later, a **message back to the original caller**.

Example:

- Client calls registerCallback(callbackFunc) on Service.
- Later, Service calls callbackFunc() to notify Client.

Broadcast Messages:

- A **broadcast** sends a message to **multiple objects at once**.
- UML doesn't have a strict symbol for broadcast, but you can:
    - Show **one sender sending the same message to multiple receivers**.
    - Label message as broadcast or multicast.

Example:

- Server → Client1, Client2, Client3: updateStatus()

Part 1: Basic Behavioral Modeling

Behavioral modeling focuses on **how the system behaves** — the actions, workflows, states, and interactions between users and objects over time.

1. Use Cases : A **use case** describes **how a user (actor)** interacts with the system to achieve a specific goal.

by saba nehal

- It is **written from the user's point of view**, often as a short story or step-by-step interaction.

Example:
Use case: *Place Order*

- Actor: Customer
- Steps: Browse → Add to Cart → Checkout → Confirm Payment

2. Use Case Diagram

- A visual representation of **all use cases and actors** involved in a system.
- Shows the **functional requirements** of a system.

Elements:

- **Actors**: External entities (users or other systems).
- **Use cases**: Ovals representing user goals.
- **Associations**: Lines showing interactions.
- **Relationships**: Include (<<include>>), Extend (<<extend>>)

Purpose: Helps understand what functionalities the system offers.

3. Activity Diagram

- Shows the **workflow** or **sequence of actions** in a process.
- Useful to model **business logic**, **algorithms**, or **procedures**.

Elements:

- Start node, activities (rounded rectangles), decisions (diamonds), arrows (flow), end node.

Example:
Login → Check credentials → If valid → Show dashboard → Else → Error message

4. State Machine Diagram (State Chart)

- Describes how an **object behaves in response to events**, depending on its **state**.
- Useful for systems with complex **state transitions**.

by saba nehal

Elements:

- States (rounded boxes)
- Transitions (arrows with events/conditions)
- Entry/exit actions

Example:
Order State: New → Confirmed → Shipped → Delivered → Returned

5. Process and Thread

- **Process**: An independent program in execution. Has its own memory space.
- **Thread**: A lightweight subprocess, shares memory with other threads of the
- same process.

In modeling:

- Helps show **concurrent activities**, useful in **multi-threaded C++ applications**.

6. Event and Signals

- **Event**: An occurrence that can trigger a change in system behavior (e.g., button click, message received).
- **Signal**: A kind of event used to communicate between objects or components.

In UML, signals are shown in **state or sequence diagrams** to indicate transitions or interactions.

7. Time Diagram (Timing Diagram)

- Focuses on **how objects or values change over time**.
- Used in **real-time and embedded systems**, especially where **timing constraints** matter.

Example:
In a traffic light system: Red → Green → Yellow → Red (based on time intervals)

8. Interaction Diagram

by saba nehal

There are two main types of interaction diagrams:

- **Sequence Diagram**: Shows time-based message flow between objects.
- **Collaboration Diagram**: Focuses on structural relationships and message exchange.

Interaction diagrams help visualize **how objects work together** to complete a task.

9. Package Diagram

- Shows how the system is **organized into packages or modules**.
- Useful for **managing large C++ projects** by organizing classes logically.

Elements:

- Packages (folders)
- Dependencies (arrows between packages)
- Visibility of elements inside packages

Part 2: Architectural Modeling

Architectural modeling shows the **high-level structure** of a system, how it's **physically deployed**, and how **components** interact.

1. Component

- A **replaceable part** of a system that encapsulates implementation (like a module, DLL, or library).
- In C++, this might be a compiled class library, a service, or an executable.

2. Deployment

- Deployment refers to **where and how software components are deployed** onto hardware (nodes).
- Nodes could be servers, clients, embedded devices, etc.

3. Component Diagram

- Describes the **software components** and their relationships.

by saba nehal

- Shows **interfaces**, **dependencies**, and **how components interact**.

Elements:

- Components (boxes with small rectangles indicating interfaces)
- Interfaces
- Dependencies (dashed arrows)

Purpose: Visualize modularity and manage dependencies in large systems.

4. Deployment Diagram

- Shows the **physical deployment of components** onto hardware nodes.
- Useful for modeling **distributed systems**, **cloud-based architecture**, or **embedded systems**.

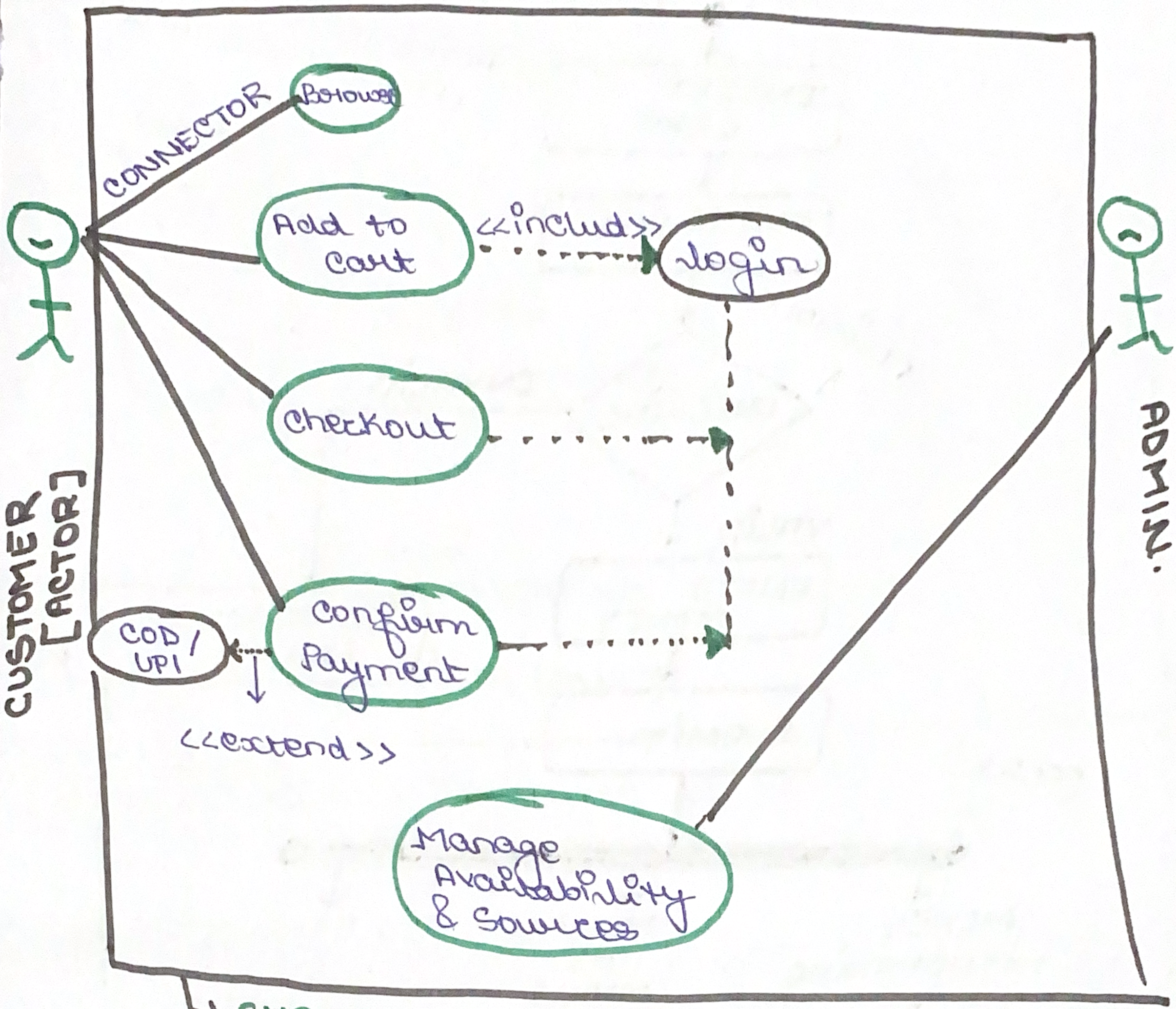
Elements:

- Nodes (physical devices or virtual machines)
- Artifacts (software pieces deployed on nodes)
- Communication paths

Example:
Web server → App Server → Database Server, showing how different software components are installed on different machines.

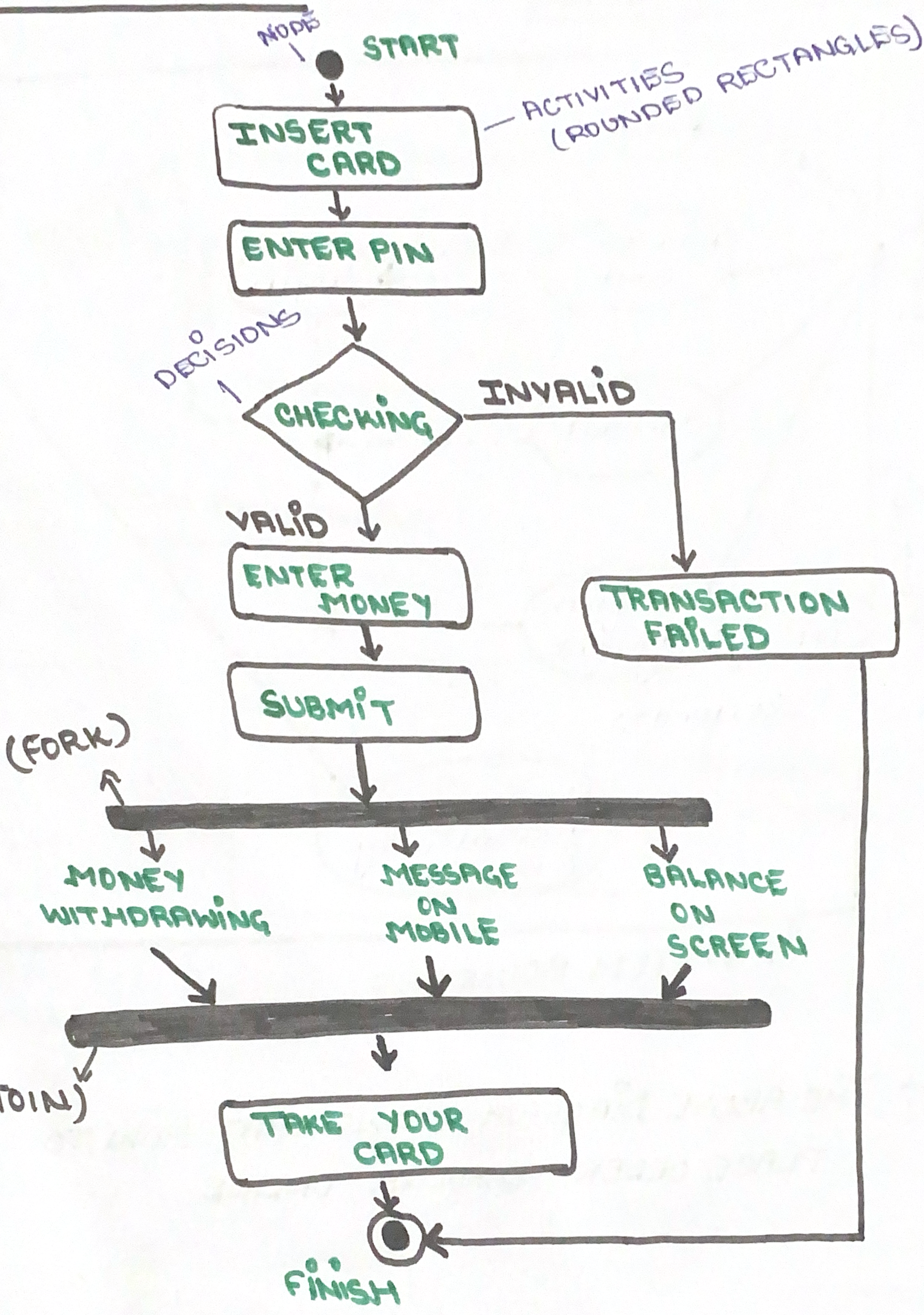
by saba nehal

USE CASE DIAGRAM



→ THE ABOVE DIAGRAM SHOWS THAT HOW TO PLACE ORDER THROUGH ONLINE

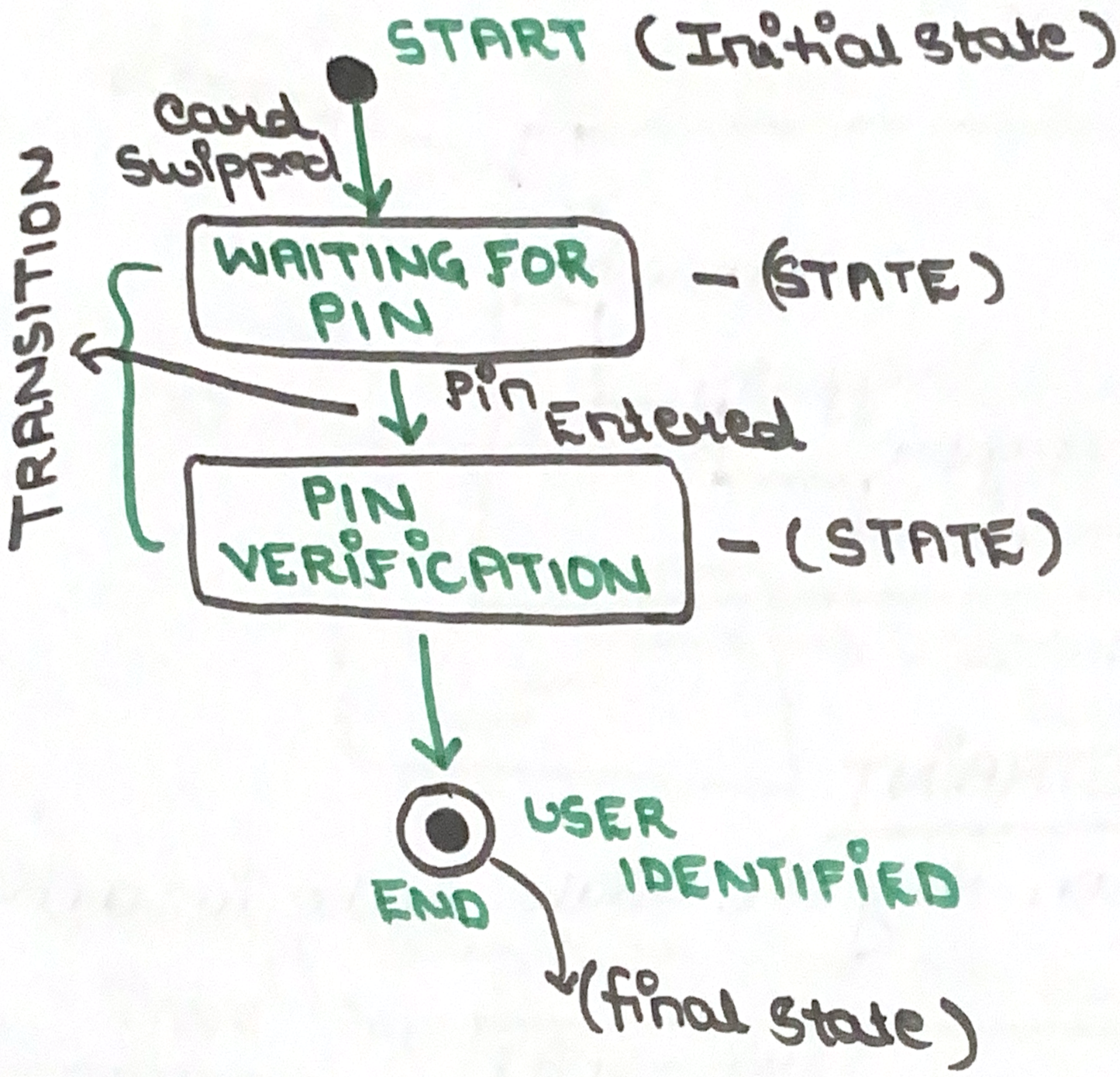
ACTIVITY DIAGRAM



THE ABOVE ACTIVITY DIAGRAM SHOWS HOW TO WITHDRAW MONEY BY USING ATM.

• STATE MACHINE DIAGRAM

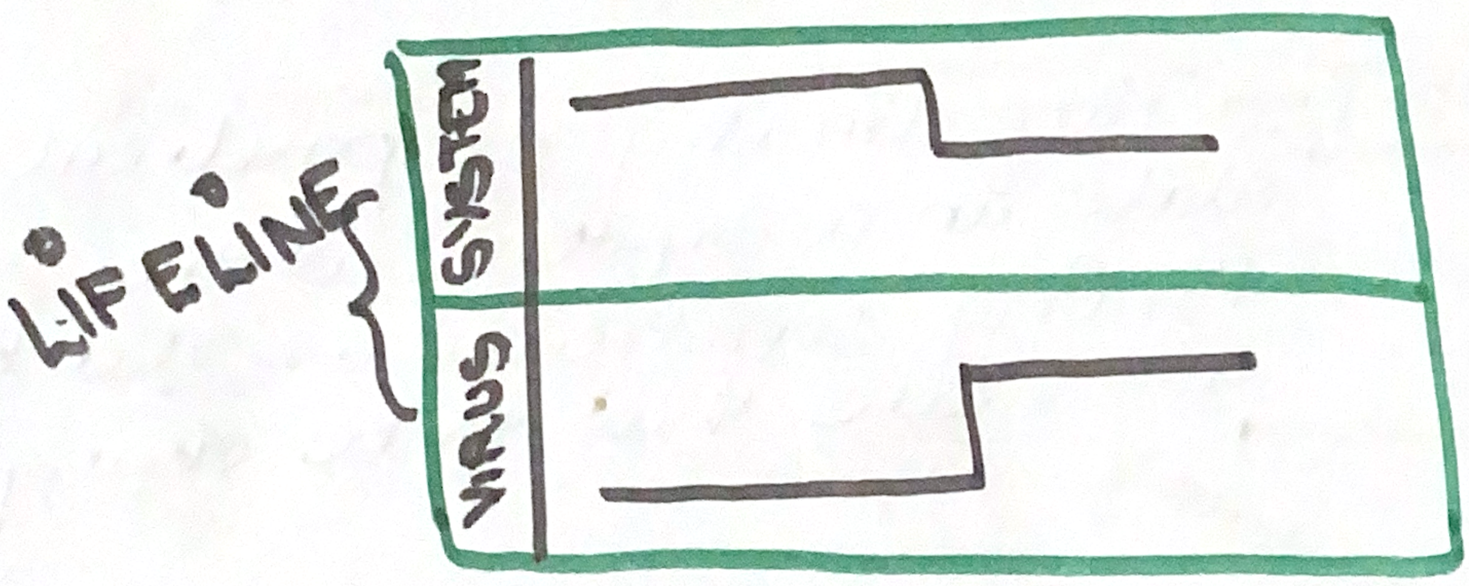
Let's take an Example of User verification (ATM)



• **TIME DIAGRAM** : It focuse on specific time when messages are sent between objects.

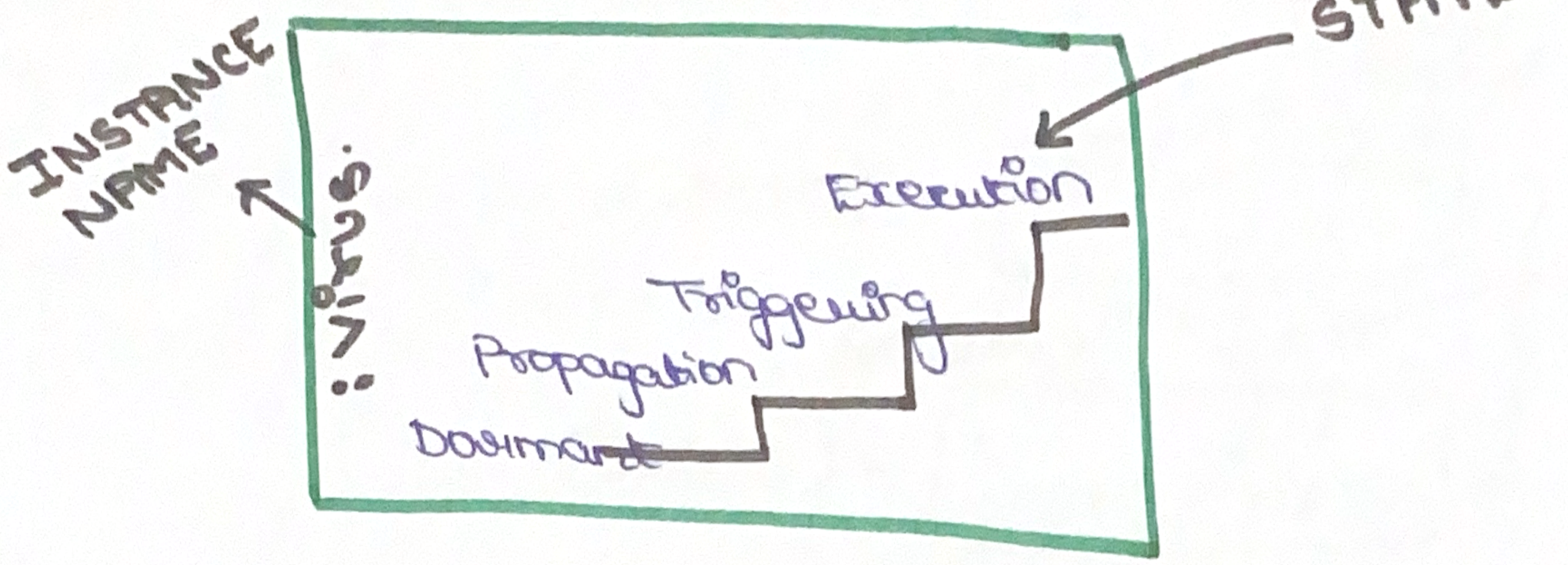
① **Lifeline** – lifeline are the object which holds the states. They are similar to lifelines used in sequence diagrams.

Graphical Notation



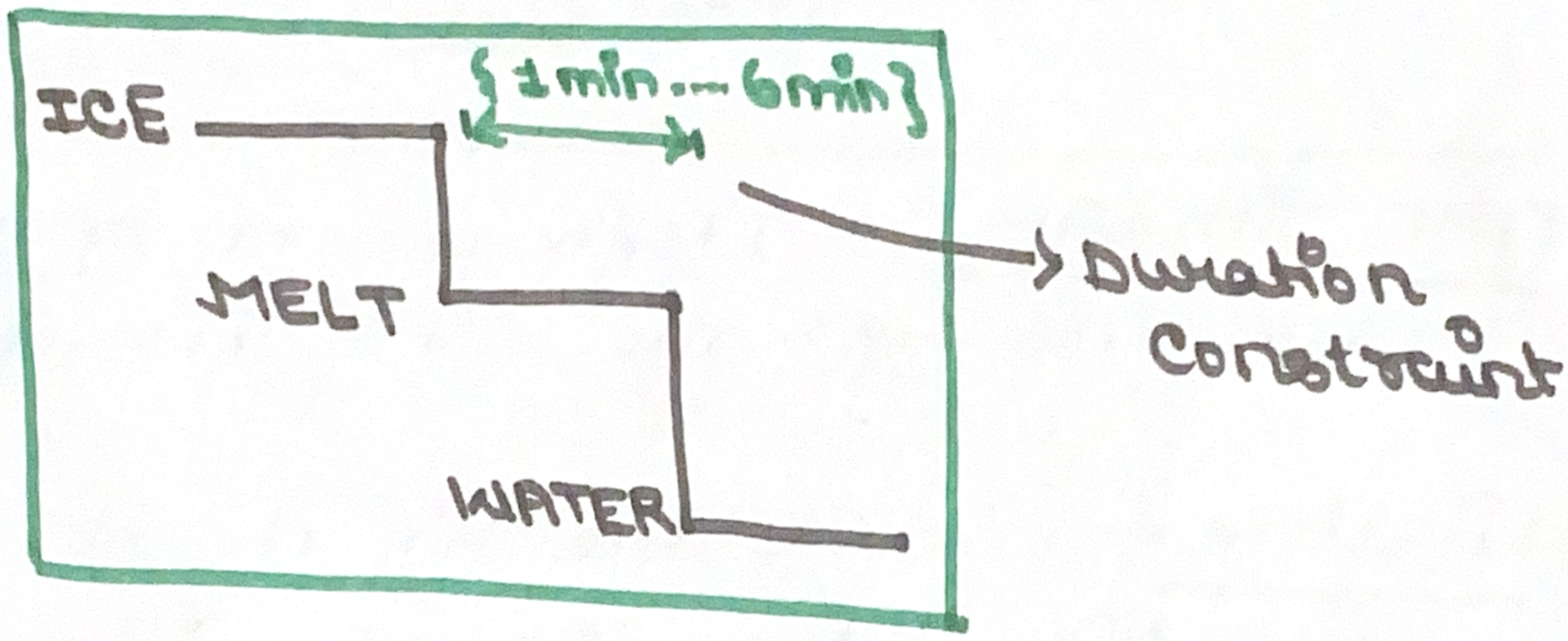
Here Lifeline representing instance of a system and Virus.

② STATE OR CONDITION → State refers to the behavior of the system at a particular instance of time.



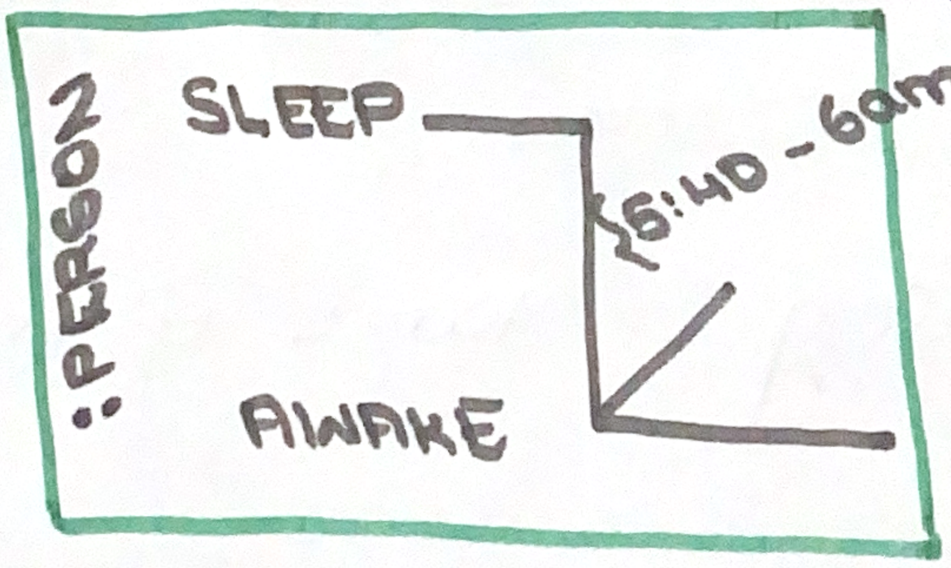
③ DURATION CONSTRAINT

The time taken by the state to be in active in the state.



Here Ice should melt into water & it will take 1 to 6 minutes.

④ TIME CONSTRAINT → Time limit for a particular state in a system when timing constraint is reached the state has to be changed.



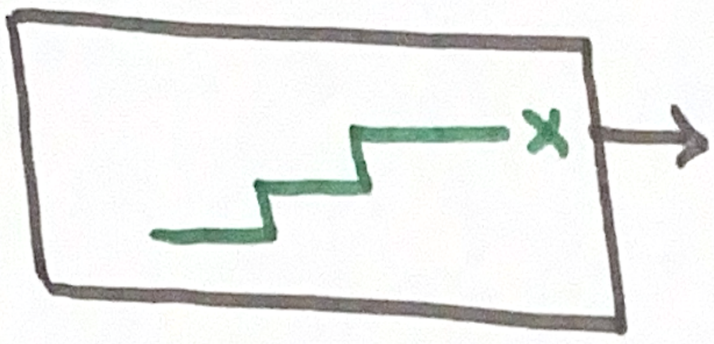
→ A person should wakeup in b/w 5:40 am & 6 am

⑤ **EVENT →** Event is/are responsible for changing states in a system.

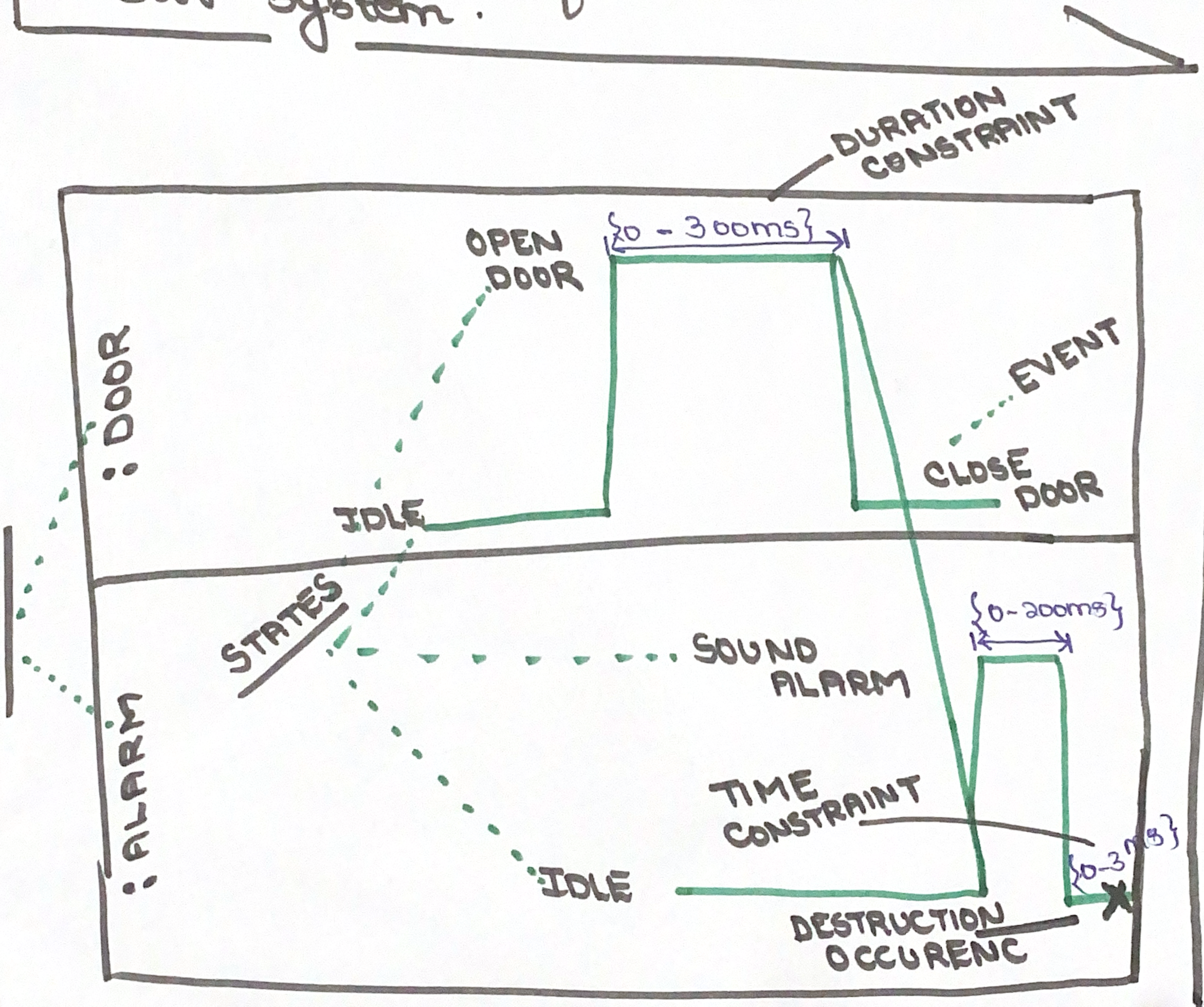
⑥ **DESTRUCTION OCCURENCE**

When an object is not a part of the system it can be destroyed.

* Destructed objects are denoted by placing an (X) mark at the end of the line.



Timing Diagram for an automatic Door System.



UNIT 3 (OBJECT ORIENTED ANALYSIS)

Object Design

Object design is the detailed refinement phase of OOD.

It includes:

- Choosing appropriate **data structures**
- Defining **method signatures**
- Specifying **visibility (public/private/protected)**
- Elaborating **interfaces and abstract classes**
- Making detailed decisions for **performance, memory usage, error handling**, etc.

Object Design is where conceptual classes become concrete definitions.

Combining Three Models

In OOA/OOD, three primary models are usually built:

a. Object Model

Represents **static structure**, including:

- Classes
- Attributes
- Methods
- Relationships

b. Dynamic Model

Represents **behavior over time**, including:

- State diagrams
- Sequence diagrams
- Activity diagrams

c. Functional Model

Represents **data transformations**, including:

- Use cases
- Data flow models

by saba nehal

- System functions

Combining these three models ensures that the system is consistent:
• The object model provides structure
• The dynamic model shows behavior
• The functional model describes system responsibilities

Together they form a complete design.

Designing Algorithms

During object design, algorithms must be created or chosen for operations carried out by objects.

Examples:

- Searching, sorting, and filtering data
- Navigation through object structures(decide traversal path)
- Resource management (e.g., memory, queues)
- Business logic algorithms(calculations,tax deduction,interest calculations)

This step focuses on **efficiency, clarity, and correctness**.

Design Optimization

Design optimization aims to improve:

a. Performance

- Reducing computation time
- Using efficient data structures
- Caching repeated computations

b. Memory usage

- Avoiding duplicate objects
- Using compact representations

c. Maintainability

- Minimizing coupling(dependency of one class on another class)
- Improving cohesion()

d. Reusability

- Applying design patterns

by saba nehal

- Generalizing classes and interfaces

Optimization must not reduce clarity .

Implementation of Control

This refers to how control flows in the system, such as:

- **Method invocations** between objects(one obj how to call other obj method)
- **Event handling(**any special event from where we take action**)**
- **Concurrency and synchronization(**run multiple task concurrently AND safe like no conflict occur**)**
- **Error and exception handling(**if error occur there is no chances of system crash**)**

Adjustment of Inheritance

Inheritance must be refined during design to ensure it is used correctly.

This includes:

- Deciding which classes should be base classes
- Avoiding incorrectly deep inheritance trees

Overuse of inheritance can make design rigid, so adjustments are often required.

Object Representation

This focuses on how objects are represented internally, including:

- Data formats
- Memory layout
- How attributes are stored
- Access methods
- Encapsulation techniques

Choices here affect performance and flexibility.

Physical Packaging

Physical packaging organizes the system into implementation units such as:

- **Modules**
- **Packages**
- **Files**

by saba nehal

- **Deployment units**

Documenting Design Considerations

Documenting design ensures future developers understand:

- Why certain decisions were made
- Constraints and assumptions(during design what are the limits like who can access and memory limit)
- Trade-offs (e.g., performance vs. readability)(not everything is perfect,something sacrifice)
- Alternative designs considered(think another options but not in used)
- Patterns used(which pattern we used and why)

by saba nehal

UNIT 3 (OBJECT ORIENTED ANALYSIS)

Structured Analysis (SA) and Structured Design (SD)

Structured Analysis (SA)

Structured Analysis is a **top-down, process-oriented** method used to understand system requirements.

It focuses on:

- **Processes** (what the system does)
- **Data flows** (how data moves)
- **Data stores**
- **External entities**

Main tools:

- **DFD – Data Flow Diagrams**
- **ER Diagrams**
- **Data Dictionary**

Goal: A clear model of *what* the system must do.

Structured Design (SD)

Structured Design converts SA results into a **modular blueprint** for implementation.

It focuses on:

- Breaking the system into **modules**
- Designing **control structures**
- Defining module **interfaces**
- Ensuring **low coupling** and **high cohesion**

Main tools:

- **Structure Charts**
- **Module Specifications**

Goal: Define *how* the system will work internally.

Jackson Structured Development (JSD)

by saba nehal

JSD is a method created by Michael Jackson for systematic software development.
It is particularly suited for **data-driven, real-time systems**.

JSD follows 3 phases:

1. Modeling Stage

- Identify **entities** in the real world.
- Draw **entity structure diagrams** showing sequence, selection, repetition.
- Define entity **life histories** (events over time).

2. Network Stage

- Convert models into a **system structure**.
- Decide how entities interact.
- Sequence tasks using structure diagrams.

3. Implementation Stage

- Transform the structure into **program code**.
- Combine processes into a final application.

JSD emphasizes:

- Real-world modeling
- Time-ordered events
- Stepwise refinement

by saba nehal

Mapping Object-Oriented Concepts Using a Non-Object-Oriented Language

Some older languages (like **C, Pascal, Fortran**) do not support OOP features directly.

To use OOP concepts in such languages, programmers must implement them **manually**.

Below are methods for implementing OOP ideas without native support.

1.Translating Classes into Data Structures

In a non-OOP language, a **class** can be represented as a:

Structure (struct) + related functions

Example in C:

```
struct Student {
   char name[50];
   int age;
   float marks;
};


// Methods implemented separately
void setMarks(struct Student *s, float m) {
   s->marks = m;
}
```

Here:

- The **struct** stores attributes (like a class)
- Functions act like **methods**

Passing Arguments to Methods

Since methods cannot be inside a class, functions receive the "object" as an argument.

Two types:

1. Pass by value

Copies the structure — inefficient for large objects.

```
void display(struct Student s);
```

2.Pass by reference

Pass pointer to the structure — behaves like calling a method on an object.

```
void update(struct Student *s, int age);
```

This simulates:

- this pointer
- Method invocation on objects

by saba nehal

Implementing Inheritance

Non-OOP languages do not have “extends” or “inherit”.

But we can implement inheritance using:

1. **Nested Structures (struct within struct)**

```
struct Person {
   char name[50];
   int age;
};

struct Student {
   struct Person base;   // Inherited fields
   float marks;
};
```

This mimics:

- Child class (Student)
- Parent class (Person)

2. Function Sets for Parent and Child

Parent "methods":

```
void setAge(struct Person *p, int age);
```

Child "methods":

```
void setMarks(struct Student *s, float marks);
```

Implementing Associations

Associations represent relationships between objects.

In non-OOP languages, associations are implemented using:

1. Pointers

A structure can contain a pointer to another structure.

```
struct Department {
   char deptName[50];
};

struct Employee {
```

by saba nehal

```
    char name[50];
    struct Department *dept;   // Association
};
```

2.Arrays of Structures

For one-to-many associations:

struct Student students[50];

3. Linked Lists / Dynamic Memory

Useful for variable-size relationships.

Implementing Encapsulation

Encapsulation means **hiding data** and providing **controlled access**.

In non-OOP languages, we simulate this by:

1. Making data structures private to a file

Use static variables in C so other files cannot access them.

static struct Student s; // private

2. Using Getter and Setter functions

Control access to fields.

```
void setAge(struct Student *s, int age) {
    if (age > 0)
        s->age = age;
}

int getAge(struct Student *s) {
    return s->age;
}
```

This imitates **private fields** and **public methods**.

by saba nehal

Object-Oriented Programming Style

Object-oriented programming (OOP) is a style of programming that uses **objects and classes** to design and build applications.
The OOP style emphasizes:

1. Reusability

Reusability means writing code once and using it many times.

OOP supports reusability through:

- **Classes** (can be reused to create multiple objects)
- **Inheritance** (child classes reuse parent class features)
- **Polymorphism** (same interface with different implementations)
- **Libraries/frameworks**

Benefit: Saves time, reduces errors, improves productivity.

Extensibility

Extensibility means the ability to **add new features** or modify existing features **without rewriting the whole system**.

Achieved by:

- **Inheritance** (extend a class)
- **Interfaces and abstract classes**
- **Polymorphism** (add new object types easily)

Example:
Adding a new payment method in a shopping app without changing the entire system.

Robustness

Robustness means the system handles errors well and continues to work reliably.

OOP improves robustness through:

- **Encapsulation** → protects data from accidental modification
- **Exception handling** → manages runtime errors
- **Strong typing** → prevents invalid operations
- **Well-defined interfaces** → clear communication among objects

by saba nehal

Result: Fewer bugs and more stable software.

Programming in the Large

Programming in the large refers to developing **large, complex systems** with multiple modules and teams.

OOP helps by:

- Organizing code into **classes, packages, namespaces, modules**
- Promoting **modularity**
- Supporting **team collaboration**
- Enabling **maintainability** and long-term evolution

Used in big systems like:

- Operating systems
- Banking systems
- Enterprise applications

- # Procedural Programming vs. Object-Oriented Programming

Feature	Procedural Programming	Object-Oriented Programming
Basic unit	Function	Object
Focus	Step-by-step actions	Objects and interactions
Data	Global or passed as parameters	Encapsulated inside objects
Reusability	Limited	High (classes, inheritance)
Security	Low (data exposed)	High (encapsulation)
Suitability	Small tasks, scripts	Complex, real-world systems
Example languages	C, Pascal	Java, C++, Python

Procedural → **Actions first**
OOP → **Objects first**

by saba nehal

Object-Oriented Language Features

OOP languages support the following key features:

1. Classes

Blueprints for creating objects.

2. Objects

Instances of classes; represent real-world entities.

3. Encapsulation

Binding data and methods together; hiding internal details.

4. Abstraction

Showing only essential information; hiding unnecessary details.

5. Inheritance

One class inherits attributes and methods from another class.

6. Polymorphism

Same operation behaves differently depending on object type.

7. Dynamic Binding

Method call decided at runtime (late binding).

8. Message Passing

Objects communicate through method calls.

Abstraction and Encapsulation

These are core principles in OOP.

Abstraction

Definition:
Abstraction means **showing only necessary details** and hiding complexity.

Example:

- A car's **steering wheel** gives simple controls
  but hides the complex machinery inside.

How OOP achieves abstraction:

- Abstract classes

by saba nehal

- Interfaces
- Public method names
- Hiding internal logic

Purpose: Simplifies programming and reduces complexity.

Encapsulation

Definition:
Encapsulation means **bundling data and methods** into a class and **restricting direct access** to the data.

Tools for encapsulation:

- Private variables
- Public getter/setter methods
- Access specifiers (private, protected, public)

by saba nehal

HOW TO CREATE A C++ PROGRAM /CODE

SYNTAX:

```
# include<iostream.h>

Using namespace std;

Main()

{

//code

}
```

PROGRAMS

1. Write a program to print two names.
2. Write a program to enter a number.
3. Write a program to declare a variable and print it.
4. Write a program to print the function with the same name.

ENUM: An **enumeration**, or `enum`, is a user-defined type representing a set of named constant values, often used to make code more readable and type-safe compared to using plain integers .

SYNTAX OF ENUM:

```
enum enumName {
        VALUE1,
        VALUE2,
         VALUE3
};
```

CODE TO EXPLAIN ENUM:

```
#include<iostream>
using namespace std;
enum Day { SUNDAY,MONDAY,TUESDAY,WEDNESDAY,THURSDAT,FRIDAY,SATURDAY
};
int main()
{
Day today=MONDAY;
if(today==MONDAY)
{
cout<<"Today is MONDAY"<<endl;
}
return 0;
}
```

SOME IMPORTANT POINTS TO REMEMBER

1. **Default value:** First value(eg; SUNDAY) its value is zero,and then next value is 1 and again next to next is 2 and so on.

{SUN,MON,TUE,WED,THU,FRI,SAT}

OOPS WITH C++

2. **Manual value**: you also have a choice to give manual values.so indexing be like:

   enum Day {MON,TUE,WED…..}{1,5,10…}

3. **Increases readability:** Instead of using the numbers ,you have to use their meaningful names. LIKE: light==0, light==1,light==2

WHY WE USE ENUM IN OUR PROGRAM?

1. It increases the code readability.
2. It reduces error.
3. The values remain fixed.
4. Grouping of related constants by using enum.

CODE: REAL-LIFE EXAMPLE

```
#include<iostream>

using namespace std;

enum TrafficLight {RED,YELLOW,GREEN

};

void showMessage(TrafficLight light)

{

if(light==RED)

 cout<<"stop!"<<endl;

else if(light==YELLOW)

 cout<<"Ready to go"<<endl;

else if( light==GREEN)

 cout<<"GO";

}

int main()

{

TrafficLight signal=YELLOW;

showMessage(signal);

return 0;

}
```

OOPS WITH C++

Practise questions

1. **WAP in C++ to print the colour of shapes(Circle,square and diamond) using the enumeration .**
2. **WAP in C++ to find the file which is saved on your system using enum.**
3. **WAP in C++ to find the area of a rectangle using default arguments.**
4. **WAP in C++ to print the multiplication of 2 integers using function overloading.**

OOPS WITH C++

CONTROL STATEMENTS IN C++

Control statements in C++ are programming constructs that alter the normal sequential flow of program execution. They enable programs to make decisions, repeat blocks of code, and jump to different parts of the program.

There are three main types of control statements in C++:

- **Decision-Making Statements (Selection Statements):** These statements allow the program to execute different blocks of code based on whether a given condition is true or false.

1. **`if` statement**: Executes a block of code if a specified condition is true.

```
if (condition) {
        // code to execute if condition is true
    }
```

CODE:

```
#include <iostream>
using namespace std;
int main()
{
   int a = 5; //create variable and assign value to it
   if (a == 5) {
      cout < < "a is equal to 5";
   }
   return 0;
}
```

2. **if-else statement:** Executes one block of code if the condition is true and another block if the condition is false.

CODE:

```
if (condition) {
    // code to execute if condition is true
} else {
    // code to execute if condition is false
}
```

```
#include <iostream>

using namespace std;

int main()

{

   int a = 10; // create variable and assign value=10

   if (a == 5) //check condition and true so execute first block

 {
```

```
    cout << "a is equal to 5";
  }
  else  //condition false so execute else block
{
    cout << "a is not equal to 5";
  }
  return 0;
}
```

- **switch statement:** Allows for multi-way branching, executing a specific block of code based on the value of an expression.

```
CODE:     switch (expression) {
        case value1:
            // code for value1
            break;
        case value2:
            // code for value2
            break;
        default:
            // code if no match
    }
```

CODE:

```
#include <iostream>
using namespace std;

int main() {
  int day = 3;

  switch (day) {
    case 1:
      cout << "Monday";
      break;
    case 2:
      cout << "Tuesday";
```

by saba nehal

```
            break;
        case 3:
            cout << "Wednesday";
            break;
        case 4:
            cout << "Thursday";
            break;
        case 5:
            cout << "Friday";
            break;
        default:
            cout << "Weekend";
    }

    return 0;
}
```

PRACTISE QUESTION

1. **Implement simple ATM PROGRAM using switch case .**

Iterative Statements (Looping Statements): These statements allow a block of code to be executed repeatedly until a certain condition is met.

- **for loop:** Used when the number of iterations is known or can be determined beforehand.

SYNTAX:

```
for (initialization; condition; increment/decrement) {
    // code to repeat
}
```

CODE:

```
#include <iostream>
using namespace std;

int main() {
    for(int i = 1; i <= 5; i++) {
        cout << "Hello World!" << endl;
```

by saba nehal

```
    }
    return 0;
}
```

while loop: Repeats a block of code as long as a condition remains true. It is an entry-controlled loop.

```
    while (condition) {
        // code to repeat
    }
```

CODE: #include <iostream>

```
using namespace std;

int main() {
    int i = 1;   // initialization

    while (i <= 5) {   // condition
        cout << "Hello World!" << endl;  // statement
        i++;   // update
    }
    return 0;
}
```

do-while loop: Repeats a block of code at least once, and then continues as long as a condition remains true. It is an exit-controlled loop.

```
do {
        // code to repeat
    } while (condition);
```

CODE #include <iostream>

```
using namespace std;

int main() {
    int i = 1;

    do {
```

by saba nehal

```
    cout << "Hello World!" << endl;
    i++;
  } while (i <= 5);

  return 0;
}
```

Jump Statements: These statements transfer control to a different part of the program.

- break statement: Terminates the execution of the innermost loop or switch statement and transfers control to the statement immediately following it.

CODE:
```
#include <iostream>
using namespace std;
int main()
{
for(int i = 1; i <= 5; i++) {
  if(i == 3) {
    break;   // loop STOPS here
  }
  cout << i << " ";
  }
  return 0;
}
```

Output:

1 2

- continue statement: Skips the rest of the current iteration of a loop and proceeds to the next iteration.

CODE:
```
#include <iostream>
using namespace std;
int main()
{
for(int i = 1; i <= 5; i++) {
  if(i == 3) {
```

by saba nehal

```
            continue;   // 3 skip
          }
          cout << i << " ";
        }   return 0;
        }
```

Output:

1 2 4 5

- goto statement: Transfers control unconditionally to a labeled statement(identifier with colon) within the same.

CODE#include <iostream>

using namespace std;

```
int main() {
   int i = 1;

   loop:              // label
   cout << i << " ";
   i++;

   if (i <= 5) {
     goto loop;       // firse label par jump
   }

   return 0;
}
```

Output:

1 2 3 4 5

return statement: Exits the current function and returns control to the calling function, optionally returning a value.

by saba nehal

```
CODE: #include <iostream>
using namespace std;
int add(int a, int b) {
   return a + b;   // function ends here and  return result
}
```

by saba nehal

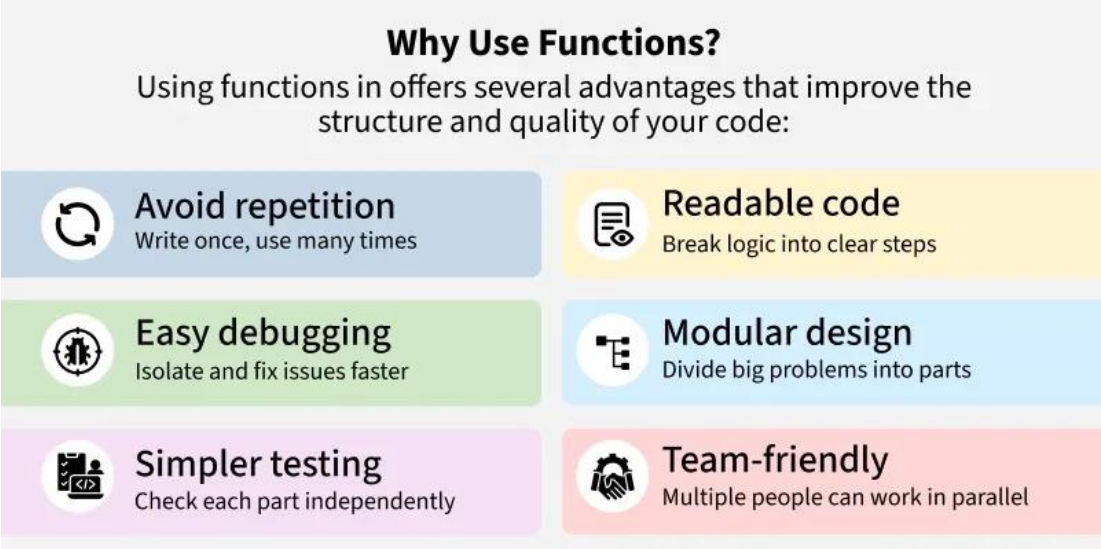
FUNCTIONS IN C++

A Function is a reusable block of code designed to perform a specific task.

It helps break large programs into smaller, logical parts.

C++ functions can take inputs (called parameters), execute a block of statements.

C++ also supports advanced features like **function overloading**, **default arguments**, and **inline functions**, which give more flexibility compared to C.



Function Syntax in C++

```
return_type function_name(parameter_list) {
    // body of the function
}
```

Each part has a specific role:

- **Return type**: Specifies what type of value the function returns. Use void if there is no return value.
- **Function name**: The name you will use to call the function.
- **Parameter list**: Inputs that the function accepts. It can be empty if no inputs are needed.
- **Function body**: The block of code that runs when the function is called.

Function Declaration vs Definition

Function Declaration introduces the function to the compiler. It tells the compiler the return type, name, and parameters but does not include the body. It ends with a semicolon.

```
// Declaration
int add(int, int);
```

Function Definition provides the actual implementation of the function.

```
int add(int a, int b) {
    return a + b;
}
```

Declaration is useful when the function is defined later in the code or in a separate file.

Types of Functions in C++

In C++, functions can be broadly categorized based on two criteria:

- **Library Functions**: These are built-in functions provided by C++ standard libraries, such as cout, sqrt(), abs(), and getline(). You can use them by including appropriate headers like <iostream>, <cmath>, or <string>.
- **User-Defined Functions**: These are functions created by the programmer to perform specific tasks in the program.

Example:program to create user defined function

```
#include<iostream.h>

Using namespace std;

Void say()//function define by user.

{

Cout<<"Hello students!!":<<endl;

}

main(){

say();  //call
```

```
say();  //call
say(); //call
say(); //call
say(); //call

}
```

Q1: WAP to define a function that prints the number.

NOTES BY, MS. SABA NEHAL

Assistant Professor

KIT, ROOMA

Friend Function:-

1:

```
#include<iostream>

using namespace std;

class Ankit;

class Ankush

{

   private:

   int money=10;

   friend void rohit(Ankit,Ankush);

};

class Ankit

{

   private:

   int money=20;

   friend void rohit(Ankit,Ankush);

};

void rohit(Ankit r1, Ankush r2)

{

   cout<<"Sum"<<r1.money+r2.money;

}

int main ()

{

   Ankush obj1;

   Ankit obj2;
```

OOPS WITH C++

```
    rohit (obj2,obj1);
    return 0;
}
```

o/p:-

```
Sum30
```

2:

```
#include<iostream>
using namespace std;

class ABC;
class XYZ {
    int x;

public:
    void get(int i) {
        x = i;
    }

    friend void sum(XYZ, ABC);
};

class ABC {
    int y;
```

OOPS WITH C++

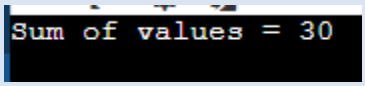
```
public:
   void get(int j) {
      y = j;
   }

   friend void sum(XYZ, ABC);
};

void sum(XYZ m, ABC n) {
   cout << "Sum of values = " << (m.x + n.y) << endl;
}

int main() {
   ABC abc;
   abc.get(10);
   XYZ xyz;
   xyz.get(20);
   sum(xyz, abc);
   return 0;
}
```

O/P:-



3:

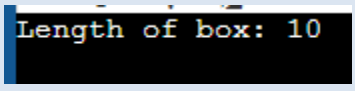
Let's see the simple example of C++ friend function used to print the length of a box.

```
#include <iostream>
```

OOPS WITH C++

```
using namespace std;
class Box
{
   private:
      int length;
   public:
      Box(): length(0) { }
      friend int printLength(Box); //friend function
};
int printLength(Box b)
{
  b.length += 10;
   return b.length;
}
int main()
{
   Box b;
   cout<<"Length of box: "<< printLength(b)<<endl;
   return 0;
}
```

O/P:-



4:Let's see a simple example when the function is friendly to two classes.

```
#include <iostream>
using namespace std;
```

OOPS WITH C++

```
class B;         // forward declarartion.
class A
{
   int x;
   public:
   void setdata(int i)
   {
      x=i;
   }
   friend void min(A,B);        // friend function.
};
class B
{
   int y;
   public:
   void setdata(int i)
   {
      y=i;
   }
   friend void min(A,B);                // friend function
};
void min(A a,B b)
{
   if(a.x<=b.y)
   std::cout << a.x << std::endl;
```

OOPS WITH C++

```
    else
    std::cout << b.y << std::endl;
}
  int main()
{
  A a;
  B b;
  a.setdata(10);
  b.setdata(20);
  min(a,b);
   return 0;
 }
```

O/P:-

In the above example, min() function is friendly to two classes, i.e., the min() function can access the private members of both the classes A and B.

5:

```
#include <iostream>
using namespace std;

class Distance {
private:
   int i;

public:
   Distance() {
```

OOPS WITH C++

```
    i = 0;
  }

  friend int add_two(Distance);
};

int add_two(Distance d) {
  d.i = d.i + 2;
  return d.i;
}

int main() {
  Distance d1;
  cout << add_two(d1);
  return 0;
}
```

o/p:-

```
2
```

6:

```
#include <iostream>
using namespace std;

class sample {
  int a, b;

public:
```

OOPS WITH C++

```
    void get_data(int i, int j) {
        a = i;
        b = j;
    }

    friend float avg(sample s);
};

float avg(sample s) {
    return float(s.a + s.b) / 2.0;
}

int main() {
    sample s;
    s.get_data(5, 6);
    cout << "\navg is " << avg(s);
    return 0;
}
```

O/P:-

```
avg is 5.5
```

OOPS WITH C++

INLINE FUNCTIONS

If a function is INLINE, then the compiler places the copy of the functions code in place of the function call.

And this can speed up the program execution.

SYNTAX OF INLINE():

```
inline return_type function_name(params)
{
//CODE
}
```

EXAMPLE: Program to implement INLINE FUNCTION

```
#include<iostream.h>
Using namespace std;
Inline int fun(int a,int b)
{
Return a+b;
}
Main(){.
Int value=fun(34,67); //calling
Cout<<value;
}
```

ADVANTAGES OF USING INLINE FUNCTION

1. Avoid repetition of codes.
2. Increase program readability.
3. Divide a complex program into many simpler problems.
4. Reduces chances of error.
5. Making modifying a program becomes easier.

SOLUTION:

```
#include <iostream>
using namespace std;
```

by saba nehal

```
inline int Max(int x, int y)
 {
 return (x > y)? x : y;
 }
 // Main function for the program
int main()
 {
 cout << "Max (30,10): " << Max(30,10) << endl;
 cout << "Max (0,100): " << Max(0,100) << endl;
cout << "Max (200,1010): " << Max(200,1010) << endl;
 return 0;
 }
```

SOLUTION:

```
#include <iostream>
using namespace std;

inline void displayNum(int num) {
   cout << num << endl;
}

int main() {
   // first function call
   displayNum(15);

   // second function call
   DisplayNum(18);

   // third function call
```

by saba nehal

```
    displayNum(66);


    return 0;
}
```

Friend function:

It is a function that is declared as a friend of a class not as a member of a class instead of that it can access private and protected member of a class.

SYNTAX: Friend return-type function name(class & reference);

ADVANTAGES OF USING FRIEND FUNCTION:

1. We can able to access the other class members in our class if we use friend function.
2. We can access the members without inheriting the class.

EXmple: program to implement the friend function

```
#include<iostream.h>
Using namespace std;
Class Ankush
{
Private:
Int money=10;
Friend void Rohit( Ankush, Ankit); //common friend that access the private property
};
Class Ankit
{
Private:
Int money=20;
Friend void Rohit(Ankit,Ankush); //common friend
};
Void Rohit(Ankit r1,Ankush r2)
{
Cout<<"sum"<<r1.money+r2.money;  //passing reference for accessing the money that borrow from ankit and ankush
Int main()
{
Ankush obj1; //create object of Ankush class
Ankit obj2; //create object of Ankit class
```

by saba nehal

```
22. Rohit (obj2,obj1);  //common friend can take both objects,obj1->r1 and obj2->r2
23. Return0;
24. }
```

MACROS: It is a preprocessor directive that defines a name or a function like macro that can be used throughout the program .

It replace the name of macro to value of macro.

Macro is defined using the '#define ' preprocessor directive.

SYNTAX:

```
#define macro _name macro _value;
```

EXAMPLE : WAP to implement macros/understand the concept of macros.(AREA OF A CIRCLE)

```
1.  #include<iostream.h>
2.  Using namespace std;
3.  #define PI 3.14        //object macro
4.  #define square(r) ((r)*(r))        //function name
5.  main(){
6.  Int r;
7.  Cout<<"enter a radius:";
8.  Cin>>r;      //r=2
9. Double area=PI*square(r);
10. Cout<<"Area of circle:"<<area;
11.  }
```

VIRTUAL FUNCTIONS:

Virtual means existing in appearance but not in reality.

Virtual functions means function() existing in class but cannot be used.

It is declared in BASE CLASS with keyword 'VIRTUAL' and redefined in DERIVED CLASS.

by saba nehal

SYNTAX: virtual return_type function_name(argument)

{

//BODY OF FUNCTIONS

}

NEED FOR VIRTUAL FUNCTION: The vital reason for having a virtual function is to implement a different functionality in the derived class.

PURE VIRTUAL FUNCTIONS:

They are virtual functions which have no definition. They start with virtual keyword and ends with equal to zero.

A pure virtual function is a function that has the Notation=0 in the declaration of that function(a function has no body).

SYNTAX: Virtual return_type fun-name()=0;

Ex-Virtual void get()=0;

EXAMPLE: pure virtual function

```
#include<iostream.h>
Using namespace std;
Class A
{
Public:
Virtual void show()=0;
Void display()
{
Cout<<"Hello:";
}
};
Class B:public A
{
Public:
```

by saba nehal

```
Void show()
{
Cout<<”Hii:”
}
};
Void main()
{
B.obj;
Ob.disp()
}
```

Program: with no virtual keyword

```
class Animal {
  public:
    void sound() {
      cout << "Animal sound\n";
    }
};

class Dog : public Animal {
  public:
    void sound() {
      cout << "Dog barks\n";
    }
};

int main() {
  Animal* a;  // Declare a pointer to the base class (Animal)
  Dog d;  // Create an object of the derived class (Dog)
  a = &d;  // Point the base class pointer to the Dog object
  a->sound(); // Call the sound() function using the pointer. Since sound() is not virtual, this
calls Animal's version
  return 0;
}
```

OUTPUT: Animal Sound

by saba nehal

Program with virtual keyword

```
class Animal {
  public:
    virtual void sound() {
      cout << "Animal sound\n";
    }
};

class Dog : public Animal {
  public:
    void sound() override {
      cout << "Dog barks\n";
    }
};

int main() {
  Animal* a;//pointer points to object
  Dog d; //object
  a = &d; //hold address or value of animal class
  a->sound(); // Outputs: Dog barks
  return 0;
}
```

OUTPUT:Dog barks

Now it works! Because sound() is virtual, the call uses the actual object's function and not just the pointer type.

by saba nehal

NOTES BY, MS. SABA NEHAL

Assistant Professor

KIT, ROOMA

OPERATORS IN C ++

An **operator** is a symbol that operates on a value to perform specific mathematical or logical computations. They form the foundation of any programming language. In C++, we have built-in operators to provide the required functionality.

An operator operates the **operands**. For example,

```
int c = a + b;
```

TYPES :-

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Ternary or Conditional Operators
7. Increment/ Decrement Operators.

1) Arithmetic Operators

These operators are used to perform arithmetic or mathematical operations on the operands. For example, '+' is used for addition, '-' is used for subtraction '*' is used for multiplication, etc.

Name	Symbol	Description	Example
Addition	+	Adds two operands	int a = 3, b = 6; int c = a+b; // c = 9
Subtraction	–	Subtracts second operand from the first	int a = 9, b = 6;

Name	Symbol	Description	Example
			int c = a-b; // c = 3
Multiplication	*	Multiplies two operands	int a = 3, b = 6; int c = a*b; // c = 18
Division	/	Divides first operand by the second operand	int a = 12, b = 6; int c = a/b; // c = 2
Modulo Operation	%	Returns the remainder an integer division	int a = 8, b = 6; int c = a%b; // c = 2

CODE:-

```
#include <iostream>
using namespace std;

int main()
{
    int a = 8, b = 3;

    // Addition operator
    cout << "a + b = " << (a + b) << endl;

    // Subtraction operator
    cout << "a - b = " << (a - b) << endl;

    // Multiplication operator
    cout << "a * b = " << (a * b) << endl;

    // Division operator
    cout << "a / b = " << (a / b) << endl;

    // Modulo operator
    cout << "a % b = " << (a % b) << endl;

    return 0;
}
```

O/P:-

a + b = 11

a - b = 5

a * b = 24

a / b = 2

a % b = 2

2) Assignment Operators

These operators are used to assign value to a variable. The left side operand of the assignment operator is a variable and the right side operand of the assignment operator is a value. The value on the right side must be of the same data type as the variable on the left side otherwise the compiler will raise an error.

Namemultiply	Symbol	Description	Example
Assignment Operator	=	Assigns the value on the right to the variable on the left	int a = 2; // a = 2
Add and Assignment Operator	+=	First adds the current value of the variable on left to the value on the right and then assigns the result to the variable on the left	int a = 2, b = 4; a+=b; // a = 6
Subtract and Assignment Operator	-=	First subtracts the value on the right from the current value of the variable on left and then assign the result to the variable on the left	int a = 2, b = 4; a-=b; // a = -2
Multiply and Assignment Operator	*=	First multiplies the current value of the variable on left to the value on the right and then assign the result to the variable on the left	int a = 2, b = 4;

Namemultiply	Symbol	Description	Example
			a*=b; // a = 8
Divide and Assignment Operator	/=	First divides the current value of the variable on left by the value on the right and then assign the result to the variable on the left	int a = 4, b = 2; a /=b; // a = 2

CODE:-

```
#include <iostream>
using namespace std;

int main()
{
    int a = 6, b = 4;

    // Assignment Operator
    cout << "a = " << a << endl;

    //  Add and Assignment Operator
    cout << "a += b is " << (a += b) << endl;

    // Subtract and Assignment Operator
    cout << "a -= b is " << (a -= b) << endl;

    //  Multiply and Assignment Operator
    cout << "a *= b is " << (a *= b) << endl;

    //  Divide and Assignment Operator
    cout << "a /= b is " << (a /= b) << endl;

    return 0;
}
```

O/P:-

```
a = 6
a += b is 10
a -= b is 6
a *= b is 24
```

```
a /= b is 6
```

3) Relational Operators:----------

These operators are used for the comparison of the values of two operands. **For example,** '>' checks if one operand is greater than the other operand or not, etc. The result returns a Boolean value, i.e., **true** or **false.**

Name	Symbol	Description	Example

Is Equal To	==	Checks if both operands are equal	int a = 3, b = 6;
			a==b;
			// returns false

Greater Than	>	Checks if first operand is greater than the second operand	int a = 3, b = 6;
			a>b;
			// returns false

Greater Than or Equal To	>=	Checks if first operand is greater than or equal to the second operand	int a = 3, b = 6;
			a>=b;
			// returns false

Less Than	<	Checks if first operand is lesser than the second operand	int a = 3, b = 6;
			a<b;
			// returns true

Less Than or Equal To	<=	Checks if first operand is lesser than or equal to the second operand	int a = 3, b = 6; a<=b;

Name	Symbol	Description	Example
			// returns true

Not Equal To	!=	Checks if both operands are not equal	int a = 3, b = 6;
			a!=b;
			// returns true

CODE:-

```
#include <iostream>

using namespace std;


int main()

{

   int a = 6, b = 4;


   // Equal to operator

   cout << "a == b is " << (a == b) << endl;//0


   // Greater than operator

   cout << "a > b is " << (a > b) << endl;//1


   // Greater than or Equal to operator

   cout << "a >= b is " << (a >= b) << endl;//1


   //  Lesser than operator

   cout << "a < b is " << (a < b) << endl;//0
```

```
    // Lesser than or Equal to operator
    cout << "a <= b is " << (a <= b) << endl;//0


    // true
    cout << "a != b is " << (a != b) << endl;//1


    return 0;
}
```

O/P:-

a == b is 0

a > b is 1

a >= b is 1

a < b is 0

a <= b is 0

a!= b is 1

4) Logical Operators

These operators are used to combine two or more conditions or constraints or to complement the evaluation of the original condition in consideration. The result returns a Boolean value, i.e., **true** or **false**.

Name	Symbol	Description	Example

Logical AND	&&	Returns true only if all the operands are true or non-zero	int a = 5, b = 2;
			a&&b;
			(5<2)&&(5>2)
			// returns FALSE

Name	Symbol	Description	Example

Logical OR	||	Returns true if either of the operands is true or non-zero	int a = 5, b = 2;
			a||b;
			(5<2)||(5>2)
			
			//returns TRUE

Logical NOT	!	Returns true if the operand is false or zero	! (5<2)= returns false //returns TRUE

CODE:-

AND:----------

```
#include <iostream>

using namespace std;


int main() {

  int x = 5;

  int y = 3;

  cout << (x > 3 && x < 10); // returns true (1) because 5 is greater than 3 AND 5 is less than 10

  return 0;

}
```

O/P:-

1

OR:----------

```
#include <iostream>

using namespace std;
```

```
int main() {

  int x = 5;

  int y = 3;

  cout << (x > 3 || x < 4); // returns true (1) because one of the conditions are true (5 is greater than 3, but 5 is not less than 4)

  return 0;

}
```

O/P-1

NOT:----------

```
#include <iostream>

using namespace std;


int main() {

  int x = 5;

  int y = 3;

  cout << (!(x > 3 && x < 10)); // returns false (0) because ! (not) is used to reverse the result

  return 0;

}
```

O/P:-

0

5) Bitwise Operators

These operators are used to perform bit-level operations on the operands. The operators are first converted to bit-level and then the calculation is performed on the operands. Mathematical operations such as addition, subtraction, multiplication, etc. can be performed at the bit level for faster processing.

Name	Symbol	Description	Example

Binary AND	&	Copies a bit to the evaluated result if it exists in both operands	int a = 2, b = 3;
			(a & b);
			//returns 2

Binary OR	|	Copies a bit to the evaluated result if it exists in any of the operand	int a = 2, b = 3;
			(a | b);
			//returns 3

Binary XOR	^	Copies the bit to the evaluated result if it is present in either of the operands but not both	int a = 2, b = 3;
			(a ^ b);
			//returns 1

Left Shift	<<	Shifts the value to left by the number of bits specified by the right operand.	int a = 2, b = 3;
			(a << 1);
			//returns 4

Right Shift	>>	Shifts the value to right by the number of bits specified by the right operand.	int a = 2, b = 3;
			(a >> 1);
			//returns 1

One's Complement	~	Changes binary digits 1 to 0 and 0 to 1	int b = 3;
			(~b);
			//returns -4

CODE:-

```
#include <iostream>
using namespace std;

int main()
{
    int a = 6, b = 4;
```

```
    // Binary AND operator
    cout << "a & b is " << (a & b) << endl;

    // Binary OR operator
    cout << "a | b is " << (a | b) << endl;

    // Binary XOR operator
    cout << "a ^ b is " << (a ^ b) << endl;

    // Left Shift operator
    cout << "a<<1 is " << (a << 1) << endl;

    // Right Shift operator
    cout << "a>>1 is " << (a >> 1) << endl;

    // One's Complement operator
    cout << "~(a) is " << ~(a) << endl;

    return 0;
}
```

O/P:-

```
a & b is 4

a | b is 6

a ^ b is 2

a<<1 is 12

a>>1 is 3

~(a) is -7
```

6) Ternary or Conditional Operators(?:)

This operator returns the value based on the condition.

```
Expression1? Expression2: Expression3
```

The ternary operator **?** determines the answer on the basis of the evaluation of **Expression1**. If it is **true**, then **Expression2** gets evaluated and is used as the answer for the expression. If **Expression1** is **false**, then **Expression3** gets evaluated and is used as the answer for the expression.

This operator takes **three operands**, therefore it is known as a Ternary Operator.

CODE:-

```
#include <iostream>
using namespace std;

int main()
{
    int a = 3, b = 4;

    // Conditional Operator
    int result = (a < b) ? b : a;
    cout << "The greatest number is " << result << endl;

    return 0;
}
```

O/P:-

```
The greatest number is 4
```

7) C++ Increment and Decrement Operators

What is a C++ increment Operator?

The C++ increment operator is a unary operator. The symbol used to represent the increment operator is (++). The increment operator increases the value stored by the variable by 1. This operator is used for Numeric values only.

There are two types of C++ increment Operator:

- Pre-Increment
- Post-Increment

1. Post-Increment operator (a++)

The postfix operator says that first use the value and then increment it. This means the value is first used up for the operation then the value is updated by 1.

CODE:-

```
// C++ Program to implement
// Post-Increment
#include <iostream>

using namespace std;

int main()
{

    int x = 5;
    cout << "Value before using post increment operator is "
            ": "
         << x << endl;
    int temp = x++;
    cout << "The value stored by temp is : " << temp
         << endl;
    cout << "After using the post increment operator is : "
         << x << endl;
```

```
    return 0;
}
```

O/P:-

```
Value before using post increment operator is : 5

The value stored by temp is : 5

After using the post increment operator is : 6
```

2. Pre-increment operator(++a)

The postfix operator says that first increment the value then use it. This means the value is increased by 1 for the operation then the value is used by the variable

Example:

```
// C++ Program to implement
// Pre-Increment
#include <iostream>

using namespace std;

int main()
{

    int x = 5;
    cout
        << "Value before using pre increment operator is : "
        << x << endl;
    int temp = ++x;
    cout << "The value stored by temp is : " << temp
         << endl;
    cout << "After using the post increment operator is : "
         << x << endl;
    return 0;
}
```

CODE:-

```
Value before using pre increment operator is : 5

The value stored by temp is : 6

After using the post increment operator is : 6
```

What is a C++ decrement Operator?

The C++ decrement operator is a unary operator. The symbol used to represent the increment operator is (–). The decrement operator decreases the value stored by the variable by 1. This operator is used for Numeric values only.

There are two types of C++ decrement Operator:

- Post-decrement operator
- Pre-decrement operator

1. Post-decrement operator (a–):

The postfix operator says that first use the value and then decrease it. This means the value is first used up for the operation then the value is decreased by 1.

Example:

```
// C++ Program to implement

// Post-Decrement

#include <iostream>



using namespace std;



int main()

{



    int x = 5;

    cout << "Value before using post decrement operator is "
```

```
            ": "

         << x << endl;

    int temp = x--;

    cout << "The value stored by temp is : " << temp

         << endl;

    cout << "After using the post decrement operator is : "

         << x << endl;

    return 0;

}
```

O/P:-

```
Value before using post decrement operator is : 5

The value stored by temp is : 5

After using the post decrement operator is : 4
```

2. Pre-decrement operator (–a)

The postfix operator says that first decrease the value and then use it. This means the value is decreased by 1 for the operation then the value is used by the variable

CODE:-

```
// C++ Program to implement
// Pre-Decrement
#include <iostream>

using namespace std;

int main()
{

    int x = 5;
    cout
        << "Value before using pre-decrement operator is : "
        << x << endl;
```

```
    int temp = --x;
    cout << "The value stored by temp is : " << temp
         << endl;
    cout << "After using the pre-decrement operator is : "
         << x << endl;
    return 0;
}
```

O/P:-

```
Value before using pre-decrement operator is : 5
The value stored by temp is : 4
After using the pre-decrement operator is : 4
```

C++ Identifiers:

All C++ **variables** must be **identified** with **unique names**.

These unique names are called **identifiers**.

Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).

Note: It is recommended to use descriptive names in order to create understandable and maintainable code:

```
#include <iostream>

using namespace std;


int main() {

  // Good variable name

  int minutesPerHour = 60;


  // OK, but not so easy to understand what m actually is

  int m = 60;


  cout << minutesPerHour << "\n";

  cout << m;

  return 0;

}
```

The general rules for naming variables are:

- Names can contain letters, digits and underscores
- Names must begin with a letter or an underscore (_)
- Names are case-sensitive (`myVar` and `myvar` are different variables)
- Names cannot contain whitespaces or special characters like !, #, %, etc.
- Reserved words (like C++ keywords, such as `int`) cannot be used as names

C++ Variables

Variables are containers for storing data values.

In C++, there are different **types** of variables (defined with different keywords), for example:

- `int` - stores integers (whole numbers), without decimals, such as 123 or -123
- `double` - stores floating point numbers, with decimals, such as 19.99 or -19.99
- `char` - stores single characters, such as 'a' or 'B'. Char values are surrounded by single quotes
- `string` - stores text, such as "Hello World". String values are surrounded by double quotes
- `bool` - stores values with two states: true or false

Declaring (Creating) Variables

To create a variable, specify the type and assign it a value:

Syntax

```
type variableName = value;
```

Where *type* is one of C++ types (such as `int`), and *variableName* is the name of the variable (such as **x** or **myName**). The **equal sign** is used to assign values to the variable.

To create a variable that should store a number, look at the following example:

Example

Create a variable called **myNum** of type `int` and assign it the value **15**:

```
#include <iostream>

using namespace std;



int main() {

  int myNum = 15;
```

```
  cout << myNum;

  return 0;

}
```

- # Constants

When you do not want others (or yourself) to change existing variable values, use the `const` keyword (this will declare the variable as "constant", which means **unchangeable and read-only**):

Example

```
#include <iostream>

using namespace std;


int main() {

  const int myNum = 15;

  myNum = 10;

  cout << myNum;

  return 0;

}
```

C++ Operators

Operators are used to perform operations on variables and values.

In the example below, we use the `+` **operator** to add together two values:

Example

```
#include <iostream>

using namespace std;
```

```
int main() {
  int x = 100 + 50;
  cout << x;
  return 0;
}
```

C++ divides the operators into the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Ternary operators
- Bitwise operators

Arithmetic Operators

Arithmetic operators are used to perform common mathematical operations.

Operator	Name	Description	Example
+	Addition	Adds together two values	x + y
-	Subtraction	Subtracts one value from another	x - y
*	Multiplication	Multiplies two values	x * y
/	Division	Divides one value by another	x / y

%	Modulus	Returns the division remainder	x % y
++	Increment	Increases the value of a variable by 1	++x
--	Decrement	Decreases the value of a variable by 1	--x

EXAMPLE OF ARITHMETIC OPERATOR:

```
#include <iostream>

using namespace std;


int main() {

 int x = 5;

 int y = 3;

 cout << x + y;

 return 0;

}
```

Assignment Operators

Assignment operators are used to assign values to variables.

In the example below, we use the **assignment** operator (=) to assign the value **10** to a variable called **x**:

Example

```
#include <iostream>

using namespace std;
```

```
int main() {
  int x = 10;
  cout << x;
  return 0;
}
```

CODE: The **addition assignment** operator (+=) adds a value to a variable:

```
#include <iostream>
using namespace std;

int main() {
  int x = 10;
  x += 5;
  cout << x;
  return 0;
}
```

A list of all assignment operators:

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3

*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Comparison Operators

Comparison operators are used to compare two values (or variables). This is important in programming, because it helps us to find answers and make decisions.

The return value of a comparison is either `1` or `0`, which means **true** (1) or **false** (0).

These values are known as **Boolean values**.

In the following example, we use the **greater than** operator (`>`) to find out if 5 is greater than 3.

A list of all comparison operators:

Operator	Name	Example
==	Equal to	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

CODE:

```
#include <iostream>
using namespace std;

int main() {
  int x = 5;
  int y = 3;
  cout << (x == y); // returns 0 (false) because 5 is not equal to 3
  return 0;
}
```

Logical Operators

As with comparison operators, you can also test for **true** (`1`) or **false** (`0`) values with **logical operators**.

Logical operators are used to determine the logic between variables or values:

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	x < 5 && x < 10
||	Logical or	Returns true if one of the statements is true	x < 5 || x < 4
!	Logical not	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)

CODE:
```
#include <iostream>

using namespace std;


int main() {
  int x = 5;
  int y = 3;
  cout << (x > 3 && x < 10); // returns true (1) because 5 is greater than 3 AND 5 is less than 10
  return 0;
}
```

TERNARY OPERATOR

The ternary operator in C++ is a conditional operator that evaluates a condition and returns one of two values based on whether the condition is true or false.

It is represented by the symbols ? and : and is also known as the "conditional operator."

Syntax of the Ternary Operator (? :) in C++

The ternary operator in C++ uses a simple formula: condition ? expression1 : expression2.
Here's what each part means:

- **Condition:** This is a test that returns either true or false.
- **Expression1:** This is what the program should do if the condition is true.
- **Expression2:** This is what the program should do if the condition is false.

Here is how you use it in code

```
int a = 10, b = 5;
int result;
// Using the ternary operator to assign the greater value.
result = (a > b) ? a : b;
// This will output: The greater number is 10
std::cout << "The greater number is " << result << std::endl;
```

Example of Ternary Operator in C++

Imagine you need to determine the smallest of three numbers. You could use an if-else structure, but the ternary operator makes your code shorter and more straightforward. Here's how you might write this:

```
int x = 34, y = 20, z = 10;
int smallest;
// Using the ternary operator to find the smallest number
smallest = (x < y) ? ((x < z) ? x : z) : ((y < z) ? y : z);
// This will output: The smallest number is 10
std::cout << "The smallest number is " << smallest << std::endl;
```

In the code above, the ternary operator is nested to compare three values. The expression (x < y) ? ((x < z) ? x : z) : ((y < z) ? y : z) first checks if x is less than y. If true, it then checks if x is also less than z. If x is smaller than both, x is the smallest. If not, z is the smallest. If x is not less than y, it checks between y and z.

C++ Bitwise Operators

In C++, bitwise operators perform operations on integer data at the individual bit-level. These operations include testing, setting, or shifting the actual bits. For example,

```
a & b;
a | b;
```

Here is a list of 6 bitwise operators included in C++.

Operator	Description
&	Bitwise AND Operator
|	Bitwise OR Operator
^	Bitwise XOR Operator
~	Bitwise Complement Operator
<<	Bitwise Shift Left Operator
>>	Bitwise Shift Right Operator

These operators are necessary because the Arithmetic-Logic Unit (ALU) present in the computer's CPU carries out arithmetic operations at the bit-level.

1. C++ Bitwise AND Operator

The **bitwise AND** `&` operator returns **1** if and only if both the operands are **1**. Otherwise, it returns **0**.

The following table demonstrates the working of the **bitwise AND** operator. Let **a** and **b** be two operands that can only take binary values i.e. **1 and 0**.

a	b	a & b
0	0	0
0	1	0
1	0	0
1	1	1

> **Note:** The table above is known as the "Truth Table" for the **bitwise AND** operator.

Let's take a look at the **bitwise AND** operation of two integers 12 and 25:

```
12 = 00001100 (In Binary)


25 = 00011001 (In Binary)


//Bitwise AND Operation of 12 and 25


     00001100
&    00011001
     _________
```

```
    00001000  = 8 (In decimal)
```

Example 1: Bitwise AND

```cpp
#include <iostream>
using namespace std;

int main() {
    // declare variables
    int a = 12, b = 25;

    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    cout << "a & b = " << (a & b) << endl;

    return 0;
}
```

2. C++ Bitwise OR Operator

The **bitwise OR** `|` operator returns **1** if at least one of the operands is **1**. Otherwise, it returns **0**.

The following truth table demonstrates the working of the **bitwise OR** operator. Let **a** and **b** be two operands that can only take binary values i.e. **1 or 0**.

a	b	a | b
0	0	0
0	1	1
1	0	1
1	1	1

Let us look at the **bitwise OR** operation of two integers **12** and **25**:

```
12 = 00001100 (In Binary)

25 = 00011001 (In Binary)



Bitwise OR Operation of 12 and 25

    00001100

|   00011001

    _________

    00011101  = 29 (In decimal)
```

Example 2: Bitwise OR

```
#include <iostream>

int main() {
    int a = 12, b = 25;

    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    cout << "a | b = " << (a | b) << endl;

    return 0;
}
```

C++ Bitwise XOR Operator

The **bitwise XOR** `^` operator returns **1** if and only if one of the operands is **1**. However, if both the operands are **0**, or if both are **1**, then the result is **0**.

The following truth table demonstrates the working of the **bitwise XOR** operator. Let **a** and **b** be two operands that can only take binary values i.e. **1 or 0**.

a	b	a ^ b
0	0	0
0	1	1
1	0	1
1	1	0

Let us look at the **bitwise XOR** operation of two integers 12 and 25:

```
12 = 00001100 (In Binary)

25 = 00011001 (In Binary)


Bitwise XOR Operation of 12 and 25

    00001100

^   00011001

    ________

    00010101  = 21 (In decimal)
```

Example 3: Bitwise XOR

```
#include <iostream>

int main() {
```

```
    int a = 12, b = 25;

    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    cout << "a ^ b = " << (a ^ b) << endl;

    return 0;
}
```

TYPE CASTING IN C++

C++ Type Conversion

C++ allows us to convert data of one type to that of another. This is known as type conversion.

There are two types of type conversion in C++:

1. Implicit Conversion
2. Explicit Conversion (also known as Type Casting)

Implicit Type Conversion

The type conversion that is automatically done by the compiler is known as implicit type conversion.

This type of conversion is also known as automatic conversion.

Example 1: Conversion From int to double

```
// Working of implicit type-conversion

#include <iostream>
```

```
using namespace std;

int main() {
   // assigning an int value to num_int
   int num_int = 9;

   // declaring a double type variable
   double num_double;

   // implicit conversion
   // assigning int value to a double variable
   num_double = num_int;

   cout << "num_int = " << num_int << endl;
   cout << "num_double = " << num_double << endl;

   return 0;
}
```

C++ Explicit Conversion

When a programmer manually changes data from one type to another, this is known as **explicit conversion**. This type of conversion is also known as **type casting**.

There are three major ways in which we can use explicit conversion in C++:

1. C++ named casts
2. C-style type casting (also known as **cast notation**)
3. Function notation (also known as **old C++ style type casting**)

For example,

```
// initializing int variable
int num_int = 26;

// declaring double variable
double num_double;
```

```
// converting from int to double
num_double = (double)num_int;
```

C++ Virtual Functions and Function Overriding

A virtual function is a member function in the base class that we expect to redefine in derived classes.
For example,

```
class Base {
   public:
    void print() {
        // code
    }
};

class Derived : public Base {
   public:
    void print() {
        // code
    }
};
```

The `print()` method in the `Derived` class shadows the `print()` method in the `Base` class.
However, if we create a pointer of `Base` type to point to an object of `Derived` class and call the `print()` function, it calls the `print()` function of the `Base` class.

In other words, the member function of `Base` is not overridden.

```
int main() {
    Derived derived1;
```

```
    Base* base1 = &derived1;

    // calls function of Base class
    base1->print();

    return 0;
}
```

To avoid this, we declare the `print()` function of the `Base` class as virtual by using the `virtual` keyword.

```
class Base {
   public:
    virtual void print() override {
        // code
    }
};
```

If the virtual function is redefined in the derived class, the function in the derived class is executed even if it is called using a pointer of the base class object pointing to a derived class object. In such a case, the function is said to be overridden.

```
CODE: #include <iostream>
using namespace std;

class Base {
   public:
    virtual void print() {
        cout << "Base Function" << endl;
    }
};

class Derived : public Base {
   public:
    void print() override {
        cout << "Derived Function" << endl;
    }
};

int main() {
    Derived derived1;
```

```
    // pointer of Base type that points to derived1
    Base* base1 = &derived1;

    // calls member function of Derived class
    base1->print();

    return 0;
}
```

Function Overloading

With **function overloading**, multiple functions can have the same name with different parameters:

Example

```
int myFunction(int x)
float myFunction(float x)
double myFunction(double x, double y)
```

CODE: Consider the following example, which have two functions that add numbers of different type:

```
#include <iostream>

using namespace std;


int plusFuncInt(int x, int y) {

  return x + y;

}


double plusFuncDouble(double x, double y) {
```

```
  return x + y;
}

int main() {
  int myNum1 = plusFuncInt(8, 5);
  double myNum2 = plusFuncDouble(4.3, 6.26);
  cout << "Int: " << myNum1 << "\n";
  cout << "Double: " << myNum2;
  return 0;
}
```

MACROS IN C++:

A **macro** is a label defined in the source code that is replaced by its value by the preprocessor before compilation.

Macros are initialized with the `#define` preprocessor command and can be undefined with the `#undef` command.

There are two types of macros: object-like macros and function-like macros.

Object-Like Macros

These macros are replaced by their value in the source code before compilation. Their primary purpose is to define constants to be used in the code.

Example

In the following example, `PI` is defined as an object-like macro:

```cpp
#include <iostream>

using namespace std;

#define PI 3.1416

int main() {
  float radius = 3;
  float area;

  area = PI * radius * radius;

  cout << "Area is " << area;

  return 0;
}
```

Function-Like Macros

These macros behave like functions, in that they take arguments that are used in the replaced code.

Example

In the following example, `AREA` is defined as a function-like macro. Note that other macros can be used in defining a subsequent macro. The inner macro is replaced by its value before the outer macro is replaced.

```cpp
#include <iostream>

using namespace std;

#define PI 3.1416
#define AREA(r) r * r * PI

int main() {
  float radius = 5;
  float result;
```

```
  result = AREA(radius);

  cout << "Area is " << result;

  return 0;
}
```

Call by Value in C++

In this method, the value of the `actual parameter` is passed to the corresponding `formal parameter` of the called function.
This technique ensures that the original data i.e. `actual parameter` does not get altered by the called function, as the called function deals only with a **copy of that data**.

When to use Call by Value in C++

- When you do not want to change the actual parameters of the function.
- When you want to make copies of the data instead of the actual data.
- When space is not an issue.
- Usually, when you do not deal with recursion or backtracking.

Advantages of Using Call by Value in C++

- This method does not change the original variable. In other words, it is preserving data.
- Whenever a function is called, it does not ever impact the actual contents of the actual arguments.
- Here, the value of actual arguments passes to the formal arguments. Therefore, any changes made in the formal argument do not impact the real cases.

Call by Reference in C++

In this method, instead of passing the values of the `actual parameters` to the formal ones, the **addresses** of the `actual parameters` are passed.

Therefore, the modification by the called function to the `formal parameters` affects the `actual parameters` as well.

When to use Call by Reference in C++

- When the programmer wants to modify the original value of a variable within a function and have those modifications reflected in the calling code.
- Suppose, you have a large object (e.g., a complex data structure or a class instance) and you want to avoid the overhead of copying it.
- When you need to modify the function parameters themselves, rather than just the values they represent.
- when working with large data, since passing by reference does not create a copy of the data. It increases efficiency.

EXAMPLE:

```
void swap(int &a, int &b) { // &a and &b are references to integers
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 5, y = 10;
    swap(x, y); // Passing x and y by reference
    cout << x << " " << y << endl; // Output: 10 5
}
```

Advantages of Using Call by Reference in C++

- The function can change the value of the argument. Thus, it is very beneficial.
- It does not create duplicate data for holding values which helps in saving the memory space.
- It helps in avoiding changes that occur by mistake.
- A person who reads the code is not aware that the value can be modified in the function.

Difference between Call by Value and Call by Reference in C++

Aspect	Call by Value	Call by Reference
What is passed	Copies the actual value of the argument.	Passes a reference or memory address of the argument.
Modification of argument	Changes made to the parameter inside the function do not affect the original argument.	Changes made to the parameter inside the function directly affect the original argument.
Memory Usage	Typically consumes less memory as it creates a separate copy of the argument.	May consume more memory as it uses references to the original data.
Performance	Generally faster due to the smaller data transfer.	May be slower due to the need to access data through references.
Data Type Constraints	Suitable for simple data types (int, float, etc.).	Suitable for complex data types (arrays, objects, etc.).
Risk of unintended changes	Reduces the risk of unintentional side effects.	Increases the risk of unintentional side effects.

CALL BY ADDRESS: call by address" (using pointers) and "call by reference" are both mechanisms to pass arguments to a function, allowing the function to modify the original variables in the calling context.

However, they differ in how they are implemented and used. "Call by reference" is generally considered more convenient and less error-prone than "call by address" (using pointers).

Call by Address (using pointers):

- **Mechanism:** The address of the actual parameter is passed to the function. Inside the function, the pointer is dereferenced to access and modify the value at that memory location.
- **Example:**

```
void swap(int *a, int *b) { // *a and *b are pointers to integers
        int temp = *a;
        *a = *b;
        *b = temp;
    }

    int main() {
        int x = 5, y = 10;
        swap(&x, &y); // Passing addresses of x and y
        cout << x << "" << y << endl; // Output: 10 5
    }
```

- **Advantages:**

  Allows direct modification of the original variable, useful for returning multiple values from a function.

- **Disadvantages:**

  Requires the use of pointers and the `&` (address-of) and `*` (dereference) operators, making it more error-prone and less readable than call by reference.

NOTES BY- MS.SABA NEHAL

OBJECT ORIENTED SYSTEM DESIGN WITH C++ (BCS-501)

UNIT 5

CLASS AND OBJECTS

CLASS: A **class** is a **user-defined data type** in C++ that can hold **variables (data members)** and **functions (member functions)** together.
You can think of it as a **blueprint or design**.

- A class itself does not occupy memory.
- Memory is allocated only when we create objects of that class.

Syntax of Class:

```
class ClassName {
  // Access specifier (public/private/protected):
  data members;    // variables
  member functions; // functions
};
```

Object: An object is a real-world entity created from a class.
It is used to access the variables and functions defined inside the class.

Example:

- "Car" is a class (blueprint/design).
- "BMW, Audi, Maruti" are objects of that class.

Creating Objects:

ClassName objectName;

PROGRAM: :

```
#include <iostream>
using namespace std;

class Student //class definition
{
public:
   int age=20;             // data members
   void display() {      // member function
      cout << "Age = " << age << endl;
   }
};
```

OOP'S WITH C++

```
int main() {
    Student s1;   // object creation
    s1.age ;  //access data member through(dot operator )
    s1.display(); //call data function
    return 0;
}
```

Static Data Members in C++

Whenever we declare a data member as a static either inside or outside of a class called static data member.

- It is **declared using static keyword**.
- Only **one copy** of the static variable exists in memory, no matter how many objects are created.
- It is always initialized with zero because its default value is zero.
- Static member shared memory for all objects of the class.

Static Member Functions in C++

A static member function is a function inside a class declared with the static keyword.

- It is access only static data member.
- It is also accessible if we do not have any object of a class.

EXAMPLE: CODE TO IMPLEMENT STATIC DATA MEMBER AND STATIC MEMBER FUNCTION

```
#include <iostream>
using namespace std;
class A                    //create a class
{
int a;                     //normal data member
static int b;              //static data member
public:
A(int x, int y)            //create constructor and pass the parameters
{
a=x;                       //value of x stored in 'a'
```

OOPS WITH

```
b=y;                          //value of y stored in 'b'
}
void show()                   //normal member function
{
cout<<a<<" "<<b;
}
static void disp()            //static member function
{
cout<<b;
}
};
int A :: b=0;
int main()
{
A obj(10, 20), obj2(100,200);     //create object of a class
//obj.show();                     //function call
obj2.show();                  //function call through second object
//A :: disp();                    //accessing static data member without object
//obj.show();                     //again normal function call
}
```

OOP'S WITH C++

Constructor

1. A constructor is a special type of function which has the same name as the class name.
2. It is being automatically call at the time of object declaration.
3. The return type of the constructor is the class type.

DESTRUCTOR

1. It is a special member function that is executed automatically when an object is destroy that has been created by the constructor.
2. A destructor declaration should always begin with the tilde(~) symbol.

CODE:

```
#include <iostream>

using namespace std;


class MyClass {

private:

   int value;  // Single variable


public:

   // Constructor

   MyClass(int v) {

      value = v;

      cout << "Constructor called. Value set to " << value << endl;

   }


   // Destructor

   ~MyClass() {
```

OOP'S WITH C++

```
        cout << "Destructor called. Object is being destroyed." << endl;
    }

    // Method to print the value
    void print() {
        cout << "Value is: " << value << endl;
    }
};

int main() {
    MyClass obj(42);  // Create object with value 42
    obj.print();      // Print object's value
    return 0;
}
```

TYPES OF CONSTRUCTOR

1. **DEFAULT CONSTRUCTOR:** A constructor that accepts no parameters is called default constructor.

SYNTAX: class_name ()

```
{
    //code
}
```

CODE:

```
#include<iostream>
using namespace std;
```

OOP'S WITH C++

```
class A
{
int a;
public:
A()
{
a=100;
cout<<a;
}
};
int main()
{
A obj;
return 0;
}
```

PARAMETRIZED CONSTRUCTOR: A constructor that accepts or receives parameters is called parametrized constructor.

SYNTAX: class_name (para1,para2,…..)

```
{
 //code
}
```

CODE:

```
#include<iostream>
using namespace std;
class A
{
```

OOPS WITH C++

```
int a,b;
public:
A(int x,int y)
{
a=x;
b=y;
}
void show()
{
cout<<a<<" "<<b;

}
};
int main()
{
A obj(10,20);
obj.show();
}
```

OOP'S WITH C++

COPY CONSTRUCTOR: A constructor that is used to copy or initialize the value of one object into another object is called copy constructor.

SYNTAX: class_name(class_name &ref)

```
{
 //code
}
```

CODE:

```
#include<iostream.h>




using namespace std;
class A              //create a class
{
int a,b;             //declare variable
public:
A(int x, int y)      //create parametrized constructor
{
a=x;                  //store value of x,y in a,b
b=y;
}
A(A &ref)            //create a copy constructor
{
a=ref.a;              //to copy the value using 'ref.a' then again store the value in
                      //          variable'a'
b=ref.b;
}
void show()
{
cout<<a<<" "<<b<<endl;
}
```

OOPS WITH C++

```
};
int main()
{
A obj(10,20);
A obj2=obj;        //initialize object 1 here
obj.show();
obj2.show();
return 0;
}
```

INHERITANCE:

It allows a class to inherit the properties and behaviour from another class.

TYPES OF INHERITENCE: C++ provides five types of inheritance. they are:

- Single Inheritance
- Multiple Inheritance
- Hierarchical Inheritance
- Multilevel Inheritance
- Hybrid Inheritance (also known as Virtual Inheritance)

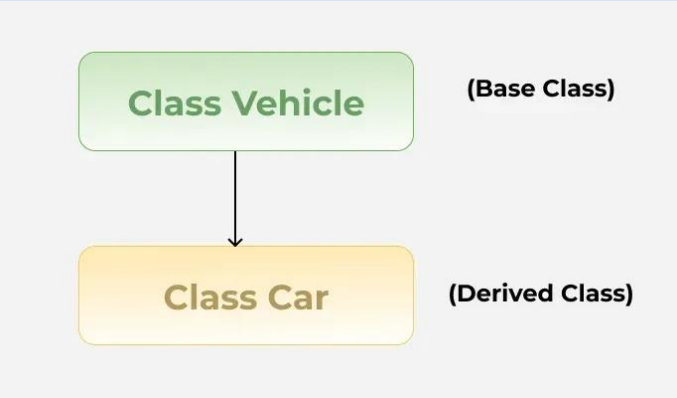
1. **Single Inheritance:** A class which contain one base class and only one derive class is called single inheritance.

SYNTAX:

```
class base
    {
        //member of base class
    };
Class derive: public base
    {
```

OOP'S WITH C++

```
        //member of derive class
    };
```



CODE

```
#include<iostream>
using namespace std;
    class base
    {
      private:
        int a,b;          //data member
        public:
       void input()
        {
        cout<<"enter values:";
        cin>>a>>b;
         }
         void show()
         {
      cout<<"a"<<"  "<<"b"<<endl;
         }
```

OOP'S WITH C++

```
        };
class derive : public base
{
  private:
     int m,n;
       public:
      void getdata()
       {
       cout<<"enter values:";
        cin>>m>>n;
}
 void display()
{
cout<<"m"<<" "<<"n"<<endl;
}
};
int main()
{
base ob;
derive ob1;

ob1.input();          //accessing base class member through the derive class
ob1.show();             //object
return 0;
}
```

OOP'S WITH C++

2. **MULTI-LEVEL INHERITANCE: :** A class which contain only one base class and multiple derive class is called multi-level inheritance.

SYNTAX: class base

```
{
    //member of base class
};
Class derive1: public base
{
    //member of derive1 class
}
Class derive2: public derive1
{
    //member of derive2 class
};
```

CODE:

```
#include<iostream>
using namespace std;
class base
{
 private:
    int a,b;            //data member
    public:
    void input()
     {
    cout<<"enter values:";
```

OOP'S WITH C++

```
                cin>>a>>b;
                 }
                 void show()
                 {
            cout<<"a"<<  "<<"b"<<endl;
              }
             };
class derive1 : public base
{
  private:
     int m,n;
       public:
      void input1 ()
       {
       cout<<"enter values:";
        cin>>m>>n;
}
 void show1()
{
cout<<"m"<<" "<<"n"<<endl;
//cout<<"a"<<" "<<"b"<<endl;
}
};
class derive2 : public base
 {
```

OOP'S WITH C++

```
  private:
     int p,q;
       public:
      void input2()
       {
       cout<<"enter values:";
        cin>>p>>q;
}
 void show2 ()
{
cout<<"p"<<" "<<"q"<<endl;
}
};

int main()
{
base ob;
derive1 ob1;
//ob1.input1();
//ob1.show1();

derive2 ob2;
ob2.input();          //base class
ob2.show();
```

OOP'S WITH C++

ob2.input2(); //derive class1

ob2.show2();

return 0;

}

3.MULTIPLE INHERITANCE: A class which contain more base class and only one derive class is called multiple inheritance.

SYNTAX: class base1

{

//data members

};

class base2

{

//member of base2 class

};

Class derive: public base1,base2

{

Member of derive class

};

OOP'S WITH C++

CODE#include<iostream>

using namespace std;

class base1

{

private:

```
    int a,b,c;              //data member
    public:
   void input()
    {
    cout<<"enter value of Base1 class:";
     cin>>a>>b;
      }
      void show()
      {
        c=a+b;
   cout<<"SUM:"<<c<< endl;
     }
   };

  class base2
{
private:
   int a,b,c;
   public:
  void input1()
     {
   cout<<"enter values Base2:";
  cin>>a>>b;
  }
void show1()
```

OOP'S WITH C++

```
{
  c=a-b;
cout<<"SUBTRACTION="<<c<<endl;
}
};
class derive : public base1,public base2
{
private:
int a,b,c;
public:
void input2()
{
cout<<"enter values of derive class:";
cin>>a>>b;
}
void show2()
{
  c=a*b;
   cout<<"MULTIPLICATION:"<<c<<endl;
}
};
int main()
{
base1 ob;
base2 ob1;
```

OOP'S WITH C++

```
derive ob2;
ob2.input2();
ob2.show2();

ob2.input();        //base1
ob2.show();

ob2.input1();       //base2
ob2.show1();
return 0;
}
```

4.HIERARCHICAL INHERITANCE: A class which contain only one base class and multiple derive class but each derive class can access base class is called hierarchical inheritance.

CODE:

```
#include<iostream>
    using namespace std;
  class base
   {
     private:
       int a,b;              //data member
       public:
      void input()
       {
```

OOP'S WITH C++

```
        cout<<"enter values for base class:";
         cin>>a>>b;
          }
          void show()
          {
        cout<<"SUM"<<a+b <<endl;
          }
        };

class derive1: public base
{
int n1;
  public:
  void input1()
{
cout<<"enter values for derive1 class:";
 cin>>n1;
}
void show1()
{
cout<<" n1"<<n1<<endl;
}
};
class derive2:public base
{
```

OOPS WITH C++

```
int n2;
  public:
  void input2()
{
cout<<"enter values for derive2 class:";
 cin>>n2;
}
void show2()
{
cout<<" n1:"<<n2<<endl;
}
};
int main()
{
derive1 ob1;
derive2 ob2;
//ob1.input();    //base class
//ob1.show();

//ob1.input1();    //derive class
//ob1.show1();

ob2.input();            //base class
ob2.show();
ob2.input2();           //derive2 class
```

OOP'S WITH C++

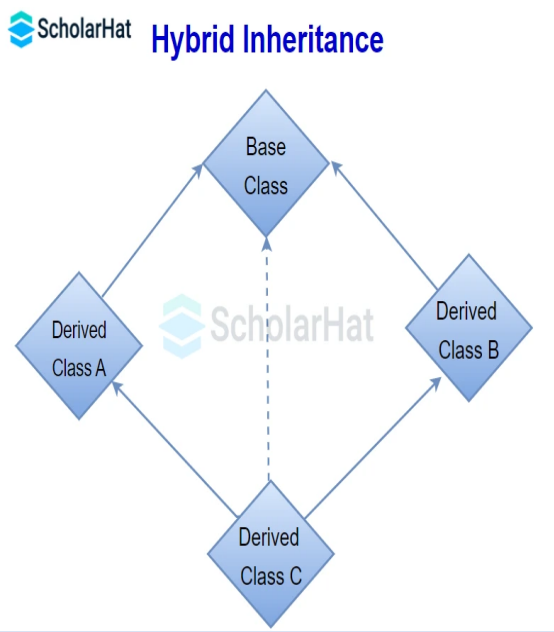
```
ob2.show2();
return 0;
}
```

5.HYBRID INHERITANCE: It is the combination of more than one type of inheritance.

```
SYNTAX: class A
            {
              //MEMBER OF BASE
    };
    Class B : virtual public A
     {
          //MEMBER OF D1 CLASS
};
Class C : virtual public A
  {
     // MEMBER OF DERIVE2 CLASS
    };

Class D : public A
```

OOP'S WITH C++



OOP'S WITH C++

```
CODE #include<iostream>
        using namespace std;
       class A
       {
         int n;
          public:
       void input()
          {
         cout<<"enter value:";
           cin>>n;
          }
       void show()
          {
         cout<<n<<endl;
         }
```

```
};
class B : virtual public A
{
 };
class C : virtual public A
{
 };
class D : public B, public C
{
};
int main()
{
A ob;
B ob1;
C ob2;
D ob3;
//ob.input();
//ob.show();

//ob1.input();
//ob1.show();

//ob2.input();
//ob2.show();
```

OOP'S WITH C++

```
ob3.input();
ob3.show();
return 0;
}
```

POINTER IN C++

A pointer is a variable that can holds the memory address of another variable.

While working with pointer we need two unary operator:

1. & =Address of operator.
2. * = Value at address operator.

EXAMPLE: int a=10;

int *ptr=&a;

Suppose address of a variable is 1000.Then,

cout<<a; //a=10 value of variable

cout<<ptr; //ptr=1000 address of variable that holds

cout<<*ptr; //*ptr=10 value at address

cout<<&a; //a=1000 address of variable

CODE:

```
#include<iostream>
using namespace std;
main(){
int a=10;
 int *p=&a;                    //pointer
// int **q=&p;                  //pointer to pointer
```

OOPS WITH C++

```
cout<<a<<endl;       //value of a
cout<<&a<<endl;      //address of a
cout<<p<<endl;       //address of variable a that pointer 'p' holds
cout<<*p<<endl;      //value at address
cout<<&p<<endl;      //address of variable'a'

// cout<<q<<endl;                    //address of p(which can hold by 'q'
                                     //      as their value)
//cout<<*q<<endl;                    //address of a
//cout<<**q<<endl;                   //value of a
// cout<<&q;                         //address of q
}
```

POINTER TO OBJECT

CODE:

```
#include<iostream>
using namespace std;
class demo
{
    public:
            void show()
             {
                cout<<"HELLO1:"<<endl;
              }
            void show1()
```

OOP'S WITH C++

```
                {
                cout<<"HELLO2:"<<endl;
               }
          };
int main()
{
demo d;
//demo *ptr=&d;       //passing address of that class object
d.show();                  //calling function by simple object

//ptr->show();      // using arrow operator to call the function
//ptr ->show1();                  //ptr==&d
return 0;
}
```

THIS POINTER: Whenever the name of instance variable and local variable both are same and if we initialize instance variable with the help of local variable then our compiler totally confused that which one is local and which one is instance varaiable.

To avoid this problem we should use 'this ' keyword.

CODE:

```
#include<iostream>
        using namespace std;
  class A
{
int a,b;
public:
```

OOP'S WITH C++

```
A(int a,int b)
{
//a=a ; b=b;        //name conflicting problem occur
this->a=a;
this->b=b;
}
void show()
{
cout<<a<<" "<<b<<endl;
}
};
int main()
{
A obj(10,20);
obj.show();
}
```

OOP'S WITH C++

NOTES BY,

MS. SABA NEHAL

ASSISTANT PROFESSOR

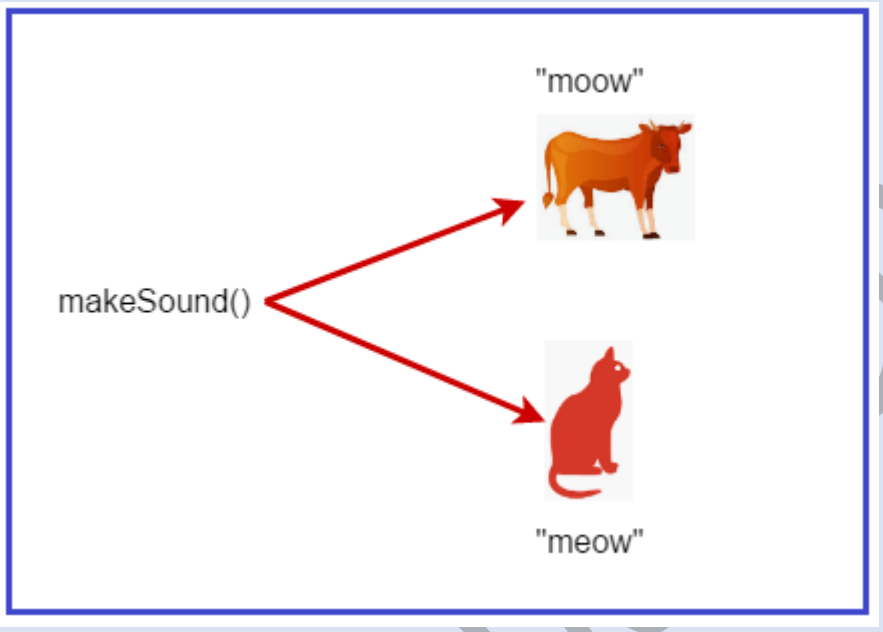
KIT, ROOMA

POLYMORPHISM

Polymorphism in C++ is a fundamental concept of Object-Oriented Programming (OOP) that means "many forms."

The word **polymorphism** means having many forms. A real-life example of polymorphism is a person who at the same time can have different characteristics.

In C++, polymorphism concept can be applied to functions. A single function can work differently in different situations.



Types of Polymorphism

Polymorphism in C++ can be classified into two types:

- **Compile-time Polymorphism**
- **Runtime Polymorphism**

1. Compile-Time Polymorphism

Also known as **early binding** and **static polymorphism**, in compile-time polymorphism, the compiler determines how the function will work depending on the context.

This type of polymorphism is achieved by function overloading.

A. Function Overloading

Function overloading is a feature of object-oriented programming where two or more functions can have the same name but behave differently for different parameters.

Such functions are said to be **overloaded;** hence, this is known as **Function Overloading**.

by sapna

CODE:

```
#include <iostream>

using namespace std;


class Math {

public:

   int add(int a, int b) {

      return a + b;

   }

   double add(double a, double b) {

      return a + b;

   }

};


int main() {

   Math m;

   cout << "Int add: " << m.add(2, 3) << endl;//function call

   cout << "Double add: " << m.add(2.5, 3.5) << endl; //function call

}
```

2. Runtime Polymorphism

Also known as **late binding** and **dynamic polymorphism.**

the function call in **runtime polymorphism** is resolved at runtime where the compiler determines which function call to bind at compilation. Runtime polymorphism is implemented using function overriding with virtual functions.

Function Overriding

Function Overriding occurs when a derived class defines one or more member functions of the base class. That base function is said to be **overridden**. The base class function must be declared as **virtual function** for runtime polymorphism to happen.

CODE: #include <iostream>

by saba nehal

```
using namespace std;

class Animal //base class
 {
public:
   virtual void sound() {   // virtual function
      cout << "Animal makes a sound" << endl;
   }
};

class Dog : public Animal //derived class
{
public:
   void sound() override {   // overriding base class function by defining
      cout << "Dog barks" << endl;
   }
};

class Cat : public Animal {
public:
   void sound() override {
      cout << "Cat meows" << endl;
   }
};

int main() {
   Animal* a;   // declare base class pointer(in this pointer we can put the address of DOG and Cat)
```

by saba nehal

```
    Dog d; //d is a Dog class obj
    Cat c;//c is a Cat class obj

    a = &d; // gives address dog d object to pointer 'a'
    a->sound();   // Output: Dog barks (decide at runtime   )

    a = &c;
    a->sound();   // Output: Cat meows (decide at runtime  )

    return 0;
}
```

A function in C++ can be overloaded in three different ways:

- By having different number of parameters.
- By having different types of parameters.
- By having both different number and types of parameters.

1. **Different Number of Parameters**

We can overload a function by changing the number of parameters that it accepts.

Example: This program demonstrates function overloading with different number of parameters. The given program has two different function definitions for the function fun with different number of parameters.

```
#include <iostream>
using namespace std;

// Two functions taking different number
// of parameters
int multiply(int a, int b);
int multiply(int a, int b, int c);
```

by saba nehal

```
int main() {

    // Calling multiply function with
    // different number of arguments
    cout << multiply(10, 2) << endl;
    cout << multiply(5, 6, 4);

    return 0;
}

// Definition of functions
int multiply(int a, int b) {
    return a * b;
}
int multiply(int a, int b, int c) {
    return a * b * c;
}
```

Different Types of Parameters

We can also change the data type of the parameters while keeping the same function name in function overloading.

Example: This program demonstrates function overloading with different data types of parameters. The given program has two different function definitions for the function fun with parameters of different data types (double and int) .

CODE:

```
#include <iostream>
using namespace std;

// Function with different arguments type
int add(int a, int b);
```

by saba nehal

```
double add(double a, double b);


int main() {


   // Calling add function with different
   // argument type
   cout << add(10, 2) << endl;
   cout << add(5.3, 6.2);


   return 0;
}


// Function definitions
int add(int a, int b) {
   return a + b;
}
double add (double a, double b) {
   return a + b;
}
```

Output

12

11.5

Different Number and Types of Parameters

Functions can also be overloaded by changing both the number and types of parameters.

Example: In the given program the function fun is overloaded and has two different function definitions with different order of parameters of different data types.

```
#include<iostream>

using namespace std;
```

by saba nehal

```
// Functions overloaded with different
// argument number and types
int add(int a, double b);
double add(double a, int b, int c);

int main() {

    // Calling both overloaded versions
    cout << add(10, 2.5) << endl;
    cout << add(5.5, 6, 12);
    return 0;
}

// Function definitions
int add(int a, double b) {
    return a + (double)b;
}
double add(double a, int b, int c) {
    return a + (double)b + (double)c;
}
```

by saba nehal

- ## Function Overloading vs Function Overriding

Definition	Same function name with different parameters.	Same function with same parameters but different function definition.

Purpose	To create multiple functions that perform similar tasks with varying arguments.	To change or extend the behavior of an inherited function.
Function Signature	Function signatures should be different.	Function signature should be similar (name, return type, parameters).
Compile-time or Runtime	Done at compile time	Done at run time
Scope	Overloaded functions are in same scope.	Overridden functions are in different scopes.

PRACTISE QUESTION

1. WAP to add 2 integers and 2 floating and 3 double numbers using function overloading.
2. What are the advantages and disadvantages of using function overloading and function over riding.
3. Demonstrate the difference between function overloading and function overriding?

Default Arguments in C++

A **default argument** is a value provided for a parameter in a function declaration that is automatically assigned by the compiler if no value is provided for those parameters in function call.

If the value is passed for it, the default value is overwritten by the passed value.

Syntax

A default argument is defined by assigning a value to a function parameter in its declaration.

```
return_type function_name (p1= v1, p2= v2, ...);
```

by saba

where **v1, v2, ...** are the default values for the parameters **p1, p2,** ... respectively.

CODE:

```
#include<iostream>
using namespace std;
void greet(string name="Guest")
{
cout<<"Hello" << name << "!" <<endl;
}
int main()
{
greet();    //no argument pass
greet("Saba"); //argument pass
return 0;
}
```

OUTPUT:

```
Hello Guest!
Hello Saba!
```

by saba nehal

Rules to Follow

There are some important rules to keep in mind when using default arguments in C++:

1. Default Values Must Be Specified in Function Declarations

The default values for parameters must be specified in the function declaration.

If a function is declared and defined separately, the default values must be in the declaration, not in definition.

```
#include<iostream>
using namespace std;
```

```
// Declaration with default argument
void func(int x = 10);

// Definition without default argument
void func(int x) {
   cout << "Value: " << x << endl;
}
int main(){
    func();  //prints valuen10
     func(50);  //prints value 50
}
```

2. Default Arguments Cannot Be Modified

Once default arguments are defined in the declaration, they cannot be modified in the function definition.

If you try to change the default value in the definition, it will result in a compilation error.

```
// Declaration
void f(int a = 10);

// This definintion will throw and error
void f(int a = 222) {
            // statements
}
```

3. Default Arguments Must Be Provided from Right to Left

In a function with multiple parameters, default values must be provided from the rightmost parameter to the left. It means that if a parameter has a default argument, all parameters to its right must also have default values.

```
// Valid
void func(int x, int y = 20);
```

by saba nehal

// Invalid, as `y` does not have a default value

void func(int x = 10, int y);

4. Ambiguity in Function Overloading

If a function containing default arguments is overloaded, then we need to make sure it is not ambiguous to the compiler, otherwise it will throw an error.

For example,

// Valid

void f(int a = 10, int b = 20);

// Will throw error as the signature is same

void f(int a = 22, int b = 2);

by saba nehal

Advantages

- **Code reusability :** Default arguments are useful when we want to increase the capabilities of an existing function.
- **Code reduction:** It helps in reducing the size of a program.
- **Flexibility :** It provides a simple and effective programming approach.
- **Consistency :** Default arguments improve the consistency of a program.

Disadvantages

- **Confusion :** If the default values are not well-documented or understood, it can lead to confusion about what arguments are being used.
- **Ambiguity :** Overloading functions with default arguments can sometimes lead to ambiguities.
- **Execution overhead :** It increases the execution time as the compiler needs to replace the omitted arguments by their default values in the function call.

ACCESS SPECIFIERS IN C++

In C++, **access specifiers** help in **hiding data** from outside the class and are a key part of **Object-Oriented Programming**.

- Which parts of your code **can** access certain variables or methods of a class.
- Which parts **cannot** directly access them.

There are 3 types of access modifiers available in C++:

1. Public
2. Private (default)
3. Protected

1. Public Specifier

Public members of a class in C++ can be accessed from anywhere in the program.
This means that both other classes and functions can directly use the data members and member functions marked as public.
To access them, the dot (.) operator is used along with the object of the class.

Code:

```
#include <iostream>
using namespace std;

class Student {
public:
    int age
=20;              // public data member(create variable)
    void display() {      // public member function
        cout << "Age = " << age << endl;
    }
};

int main() {
    Student s1;   // create object
    s1.age ;  //access public member through(dot operator )
    s1.display(); //call public function
    return 0;
}
```

2. Private Specifier

The class members declared as ***private*** can be accessed only by the member functions inside the class.

They are not allowed to be accessed directly by any object or function outside the class.
Only the member functions or the friend functions are allowed to access the private data members of the class.

CODE:

```
#include <iostream>
using namespace std;

class Student {
private:
    int age;   // private member (X direct access )

public:
    void setAge(int a) {    // public function (give value to private member)
        age = a;
    }

    void display() {       // public function (to display private member)
        cout << "Age = " << age << endl;
    }
};

int main() {
    Student s1;

    // ❌ Direct access :
    // s1.age = 20;   // ERROR

    // ✅ Access public functions :
    s1.setAge(20);     // access private member value
    s1.display();      // to display private member

    return 0;
}
```

3. Protected Specifier

The **protected** access modifier is similar to the private access modifier in the sense that it can't be accessed outside of its class unless with the help of a friend class.

by saba nehal

The difference is that the class members declared as Protected can be accessed by any subclass (derived class) of that class as well.

 Protected members as same as **private members** →
"They cannot be accessed directly by creating an object outside the class."
 Difference:

- **Private members → "They can only be accessed within the same class."**
- **Protected members → "They can be accessed within their own class and also inside the derived (child) classes created from that class."**

CODE:
```
#include <iostream>
using namespace std;

class Parent {
protected:
    int value;   // protected member
public:
    void setValue(int v)//function to access private member
 {
        value = v;//set value
    }
};

class Child : public Parent {
public:
    void showValue() {
        cout << "Value = " << value << endl; // access value
    }
};

int main() {
    Parent p;
    // ❌ Direct access :
    // p.value = 10;   // ERROR
```

by saba nehal

```
    Child c;
    c.setValue(50);   // set value through public function
    c.showValue();    // access inside the derived class

    return 0;
}
```

PRACTISE QUESTION

1. Write one C++ program to demonstrates the implementation of all three access specifiers: public, private, and protected.

by saba neha